The Bell Experiment, Monadically

Anonymous

Revised draft, November 2026

Abstract

Bell's theorem says that no classical model satisfying certain independence assumptions can reproduce the correlations that quantum mechanics predicts and experiments observe. In the CHSH form of the experiment, such models stay within 2, while quantum mechanics reaches $2\sqrt{2}$. In this paper we write the two-party Bell experiment as a small Haskell program, and find those assumptions in its structure: in what each part of the program can see, and in how the parts are composed. The boxes of the two parties compute in one monad, and the experiment that schedules and repeats their trials computes in another; a *membrane* runs the first inside the second. A run keeps the record of its trials, and both of its verdicts, the CHSH value and whether the boxes signal, are read off one table: the empirical model of that run.

In this setting, a model is classical when it has three properties. It is *blind*: the hidden strategy of the boxes is drawn without seeing the measurement settings. It is *local*: each box sees only its own setting. It is *factored*: the joint response of the boxes is the applicative pair of their local responses. Together, they give the classical bound of 2. We give them up one at a time and run what results. Without blindness, we get experimental loopholes and superdeterminism. Without factoring, we get quantum boxes, which reach $2\sqrt{2}$, and the hypothetical PR boxes, which reach 4, both without signaling. Without locality, we get boxes that rewrite a shared hidden state; whether they signal depends on how that state is sampled.

We also compare two ways to check no-signaling: reading it off the run's own table, and estimating it in separate experiments at fixed settings.

Contents

1 Introduction **3**

2 Boxes as Effectful Computations **5**

3 An Experiment in Two Layers **7**

3.1 Two Layers of Effects . . . 7

3.2 Schedulers . . . 8

3.3 One Experiment, One Table . . . 9

3.4 Testing No-Signaling . . . 11

3.5 Scheduler-Induced Loopholes . . . 13

4 The λ-Model in Code: Blind, Local, Factored **14**

4.1 Local Hidden Variables as Immutable Data . . . 15

4.2 Blind Sampling . . . 16

4.3 Superdeterminism: A Sampler That Sees the Settings . . . 19

5 Quantum Correlations in Code **21**

5.1 Qubits and Measurements . . . 22

5.2 An Entangled Pair . . . 23

5.3 Overriding `jointAB` . . . 24

5.4 CHSH Behavior and No-Signaling . . . 25

6 PR Boxes: The Algebraic Maximum without Signaling **26**
6.1 The PR Constraint . . . 26
6.2 A PR Trial Monad . . . 27
6.3 Quantum versus PR: Global Assignments . . . 28
7 Hidden States That the Boxes Rewrite **29**
7.1 The PR Box from a Hidden State . . . 29
7.2 The Bell Table from a Hidden State . . . 30
8 No-Signaling, Checked in the Run **32**
8.1 Two Ways to Check . . . 33
8.2 Why the Two Can Differ . . . 33
8.3 Every Model, Both Checks . . . 34
9 Conclusion **35**
10 Related Work **37**
11 Future Directions **38**
AI Use **38**
A Quantum Mechanics from Scratch **41**
A.1 One Qubit . . . 41
A.2 Measurements: Observables and Projectors . . . 41
A.3 Two Qubits and Entanglement . . . 43
A.4 Correlations, the CHSH Value, and No-Signaling . . . 45

1 Introduction

Bell's theorem draws one of the sharpest lines between classical and quantum models of information. Bell (1964) showed that no classical theory satisfying a natural *locality assumption* can reproduce all of the correlations that quantum mechanics predicts: systems whose behavior is determined by local information alone cannot account for certain statistical patterns.

Clauser, Horne, Shimony, and Holt (Clauser et al., 1969) recast Bell's argument in operational terms, as an inequality, now called the CHSH inequality, whose violation can be tested in the laboratory. Experiments of increasing precision followed, from those of Aspect et al. (1982) to the loophole-free tests of Hensen et al. (2015), and they have consistently confirmed the quantum predictions. The 2022 Nobel Prize in Physics was awarded to Alain Aspect, John F. Clauser, and Anton Zeilinger "*for experiments with entangled photons, establishing the violation of Bell inequalities and pioneering quantum information science.*"

The scenario behind the CHSH inequality takes no physics to describe. We are given two black boxes. Each box accepts one of two inputs, a measurement *setting*, and produces one of two outputs, an *outcome*. We assume nothing about the inside of the boxes; only their input-output behavior is observable.



Figure 1: Two black boxes f and g.

As a first approximation, we model the two boxes as deterministic functions

$$f, g \,:\, \{0,1\} \longrightarrow \{\pm 1\}.$$

Two space-like separated experimenters, Alice and Bob, each hold one box, f and g respectively (Figure 1). In one *trial* of the experiment, Alice chooses a setting a and Bob a setting b, and they observe the outcomes $(x, y) = \big(f(a), g(b)\big)$. We call the pair of settings (a, b) the *context* of the trial; there are four contexts.

To quantify the correlation between the two outcomes, we give each trial a score: the product $f(a)g(b)$, which is $+1$ when the outcomes agree and -1 when they disagree. The CHSH expression combines the scores of the four contexts:

$$\mathrm{CHSH}(f, g) = f(0)g(0) + f(0)g(1) + f(1)g(0) - f(1)g(1).$$

The minus sign on the last term is essential. We invite the reader to pause here and work out which values this expression can take when f and g are fixed functions.

❧ ❧

The four values $f(0), f(1), g(0), g(1)$ are fixed in advance, and each is ± 1. Regrouping the terms gives

$$\mathrm{CHSH}(f, g) = f(0)\big(g(0) + g(1)\big) \;+\; f(1)\big(g(0) - g(1)\big).$$

If $g(0) = g(1)$, their sum is ± 2 and their difference vanishes; if $g(0) \neq g(1)$, the sum vanishes and the difference is ± 2. Either way exactly one term survives, and its value is ± 2. For deterministic boxes, then, the CHSH expression takes only the values $+2$ and -2. Its absolute value is bounded by 2: the *classical bound*.

Two quantum surprises. The first surprise, and the one that history emphasizes, is that quantum mechanics exceeds this bound. Measuring a quantum system yields one of its possible outcomes at random, with probabilities determined by the state of the system and by the measurement chosen. Two systems are *entangled* when their joint state is not made of a state of each; the outcomes of measurements on the two can then be correlated, however far apart the systems are. Boxes that answer at random give a context no fixed score: the score xy varies from trial to trial, and the CHSH expression combines its expected values, the *correlators* $E(a, b)$,

$$\mathrm{CHSH} = E(0,0) + E(0,1) + E(1,0) - E(1,1),$$

which is $\mathrm{CHSH}(f, g)$ again for deterministic boxes, whose correlators are $E(a, b) = f(a)g(b)$. When Alice and Bob each measure one of two entangled systems, with well-chosen measurements, quantum mechanics predicts the values $\pm 2\sqrt{2}$, and experiments observe them, as statistical averages over many *repeated trials*.

The second surprise is that $2\sqrt{2}$ is as far as quantum mechanics goes: this is the Tsirelson bound (Tsirelson, 1980). Algebraically, the bound is unexpected. The CHSH expression combines four terms, each between -1 and $+1$, so its algebraic maximum is 4.

Signaling or not? Quantum mechanics cannot reach the value 4, but boxes can, in two ways. The first is trivial: boxes that communicate during a trial can coordinate their outcomes so that all four terms contribute $+1$. Such boxes violate a condition of Bell experiments called *no-signaling* (Abramsky and Brandenburger, 2011; Bell, 1964; Popescu and Rohrlich, 1994). We write $[\, c \vdash e \,]$ for the probability of e given c, and $[\, e \,]$ for that of e. No-signaling requires that one party's choice of setting does not affect the distribution of the other party's outcome:

$$[\, a, b \vdash x \,] = [\, a \vdash x \,], \qquad [\, a, b \vdash y \,] = [\, b \vdash y \,]. \tag{1}$$

In physical terms, the boxes cannot carry a message between Alice and Bob, who are space-like separated: special relativity allows none.

The second way is subtle. Popescu and Rohrlich (1994) described hypothetical correlations, now called *PR boxes*, that satisfy no-signaling exactly and still reach $\mathrm{CHSH} = 4$: they are stronger than any correlations that quantum mechanics allows.

No-signaling alone therefore does not explain the Tsirelson bound. Quantum correlations exceed the classical bound 2, and stop well short of the maximum 4 that no-signaling permits. The three bounds form a hierarchy:

$$\text{Classical } (\mathrm{CHSH} \leq 2) \;\subset\; \text{Quantum } (\mathrm{CHSH} \leq 2\sqrt{2}) \;\subset\; \text{No-Signaling } (\mathrm{CHSH} \leq 4).$$

It corresponds to strict inclusions between sets of empirical models that are studied extensively in the literature on nonlocality and contextuality (Abramsky, 2013; Abramsky and Brandenburger, 2011).

This paper. What must a model give up in order to climb this hierarchy? Bell's theorem answers with independence assumptions stated in probabilities. We answer with a program. We write the CHSH experiment in Haskell so that the boxes may use any computational effect, run it, and read the CHSH value and a no-signaling verdict off the record of the run. Bell's assumptions then become properties of the program, and each step up the hierarchy, like each experimental loophole, becomes a small, visible change to the code. That presentation is the contribution. Specifically, we give

- an executable model of a Bell experiment in two layers, in which the boxes compute in one monad and the experiment in another, joined by a *membrane* (Sections 2 and 3);
- Bell's assumptions as three properties of such programs, *blind*, *local* and *factored*, two of which have the same shape at the two layers: a pair built by applicative composition, whose components come to depend on each other when the property is given up (Sections 4 and 9);
- a zoo of models that give the properties up one at a time: experimental loopholes and superdeterminism, which are not blind (Sections 3.5 and 4); quantum boxes and PR boxes, which are not factored (Sections 5 and 6); and a shared location and hidden states that the boxes rewrite, which are not local, the latter giving the PR box and also a table that quantum mechanics realizes (Sections 2 and 7);
- for every run, both verdicts read off one table, its empirical model, and a demonstration of why: a no-signaling check made in separate experiments passes runs that signal (Section 8).

We assume a reading knowledge of Haskell, including type classes and the `Reader` and `State` monads (Hutton, 2016); library functions are explained where they first appear. We assume no background in quantum mechanics: Section 5 introduces the state vectors, projectors and Born rule that it uses, and Appendix A explains them from scratch. The complete implementation, with a test suite, is included as supplementary material. Every listing here is quoted from it, without its comments, and numbered in the margin. Every number, table and printed report of a run is written by a program that runs the models and stops if a claim of the text about them does not hold; runs that use random numbers fix the generator first, each with a seed of its own, so they are reproducible.

2 Boxes as Effectful Computations

Pure functions cannot exceed the classical bound (Section 1), so any violation requires a computational effect. Our plan is to write the CHSH experiment once, parameterized by the effect that the boxes may use inside: randomness, shared state, or something more structured. Each choice of effect is an assumption about the resources of the boxes. Seeing which effects lead to which CHSH values will isolate the assumptions behind the classical bound, those that permit the quantum violation, and those that overshoot it.

Settings and outcomes are enumerations:

```haskell
data ASetting = A0 | A1 deriving (Eq, Enum, Bounded)
data BSetting = B0 | B1 deriving (Eq, Enum, Bounded)
data Outcome = Plus | Minus deriving (Eq, Enum, Bounded)
trialValue :: (Outcome, Outcome) -> Int
trialValue (x, y) | x == y    = 1
                  | otherwise = -1
chshOf :: Num r => (ASetting -> BSetting -> r) -> r
chshOf v = v A0 B0 + v A0 B1 + v A1 B0 - v A1 B1
```

L1

`trialValue` is the score xy of a trial, and `chshOf` combines a number per context as the CHSH expression does: a score in this section, a correlator from Section 3.3 on.

The boxes are the methods of a type class, in a monad `t` that stands for whatever resources they may use:

```
class Monad t => TrialModel t where                                  L2
  localA :: ASetting -> t Outcome
  localB :: BSetting -> t Outcome
  jointAB :: ASetting -> BSetting -> t (Outcome, Outcome)
  jointAB a b = liftA2 (,) (localA a) (localB b)
```

The methods `localA` and `localB` are the two boxes, each a function of its own setting: the *local responses*. The method `jointAB` is one trial of both boxes: the *joint response*. By default it is `liftA2 (,) (localA a) (localB b)`, which runs Alice's box, then Bob's, and pairs their outcomes. An instance may override it, and the models of Sections 5 to 7 do.

Boxes as pure functions. The deterministic boxes of the introduction live in the `Identity` monad. As an example, let each box answer `Plus` at either setting. For the moment we compute the CHSH expression directly, from the score of one trial in each context:

```
instance TrialModel Identity where                                   L3
  localA _ = pure Plus
  localB _ = pure Plus
chshId :: Int
chshId = chshOf (\a b -> trialValue (runIdentity (jointAB a b)))
```

Evaluating `chshId` gives 2. The statistics of these boxes fill a table with a row for each context and a column for each pair of outcomes (Figure 2). Such tables may be viewed as *empirical models* à la Abramsky and Brandenburger (2011): they give the distribution of the joint outcome in each context. Here every entry is 0 or 1, because nothing in the boxes is random.

a	b	$(+,+)$	$(+,-)$	$(-,+)$	$(-,-)$
A0	B0	1	0	0	0
A0	B1	1	0	0	0
A1	B0	1	0	0	0
A1	B1	1	0	0	0

Figure 2: The deterministic all-plus empirical model.

Boxes with a shared location. At the other extreme, a state monad makes the value 4 easy to reach. Alice and Bob share a location. Alice's box writes her setting there and answers `Minus`. Bob's box reads the location: if the settings are `(A1, B1)`, it disagrees with Alice and answers `Plus`; otherwise it answers `Minus`. Below, `modify'` updates the location, `gets` reads it, and `evalState` runs each trial from an empty location:

```
data Shared = Shared { lastASetting :: Maybe ASetting }              L4
instance TrialModel (State Shared) where
  localA a = do
    modify' (\s -> s { lastASetting = Just a })
    pure Minus
  localB b = do
```

```
    ma <- gets lastASetting
    pure $ case (ma, b) of
      (Just A1, B1) -> Plus
      _             -> Minus
chshST :: Int
chshST = chshOf (\a b -> trialValue (evalState (jointAB a b) (Shared Nothing)))
```

Evaluating `chshST` gives 4, beyond even the quantum bound. It does so by signaling: Bob's outcome depends on Alice's setting. Nothing in the types of the methods prevented that. `localB` is never handed Alice's setting; it finds it in the state.

These two instances mark out the range of the framework. Pure boxes are too weak to account for quantum correlations, and unrestricted effects trivialize the experiment by letting the boxes communicate. The question is what lies between the two.

3 An Experiment in Two Layers

A CHSH experiment is not one trial but many, with varying settings. This section adds what repetition requires: a second layer of effects, a scheduler that chooses the settings, the record of a run with the table that is tallied from it, and the two verdicts that are read off that table.

3.1 Two Layers of Effects

Repetition gives the boxes an opportunity that a single trial does not: they can coordinate *across* trials, without communicating during any one of them. This is how the memory and timing loopholes of real Bell tests arise. To model them, we distinguish the effects inside one trial, in the monad `t`, from effects that persist across the trials of an experiment, in a second monad:

```
type Exp e = ReaderT Int e                                          L5
type RunTrial e t = forall a. t a -> Exp e a
```

An experiment is a sequence of *steps*, numbered from 0, with one trial at each. The type `Exp e` is the monad of instructions that may read the current step: an instruction of type `Exp e a` wraps a function `Int -> e a`, which, given the step i, computes an `a` with the effect `e` of the lab in which the experiment runs (`Identity` or `IO` in this paper). A value of type `RunTrial e t` embeds a trial computation `t a` into the experiment, for every result type `a`. We call such an embedding a *membrane*: it decides what passes from the experiment into a trial.

The membranes of the two models of Section 2 let nothing through. `ReaderT` makes an instruction from a function of the step; both functions ignore the step and run each trial on its own, the second from an empty location:

```
liftIdentity :: Applicative e => RunTrial e Identity                L6
liftIdentity ta = ReaderT (\_ -> pure (runIdentity ta))
```

```
runTrialState :: RunTrial Identity (State Shared)                    L7
runTrialState t = ReaderT (\_ -> Identity (evalState t (Shared Nothing)))
```

Most membranes in this paper ignore the step. It matters for boxes that adapt across trials (Section 3.5) and for strategies that vary from trial to trial (Section 4.2).

Figure 3 shows the architecture that results: two layers, and the membrane between them. The experiment layer chooses the settings with a *scheduler*, may draw a strategy for the boxes with

a *sampler* (Section 4.2), and runs the steps; the trial layer holds the boxes; the membrane runs the boxes for the experiment. At step i the settings (a_i, b_i) and a strategy λ_i flow down, and the outcomes (x_i, y_i) flow back up.

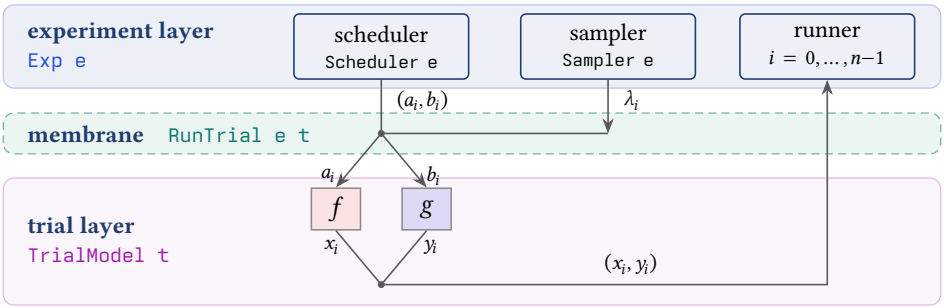


Figure 3: The architecture: two layers, and the membrane between them. The experiment layer (`Exp e`) holds the scheduler, the sampler and the runner; the trial layer (`TrialModel t`) holds Alice's box f and Bob's box g; the membrane (`RunTrial e t`) runs the boxes for the experiment layer.

3.2 Schedulers

Every experiment now needs a *scheduler*: an instruction of the experiment layer that returns the settings Alice and Bob use at the current step. Section 2 went through the four contexts once, in a fixed order: `(A0, B0)`, then `(A0, B1)`, then `(A1, B0)`, and finally `(A1, B1)`. That order was arbitrary, and we do not expect it to affect the result of the experiment. In a real Bell test, Alice and Bob choose their settings at random, and so can a scheduler:

```haskell
type Context = (ASetting, BSetting)                                    L8
type Scheduler e = Exp e Context
fixedSchedule :: Monad e => Scheduler e
fixedSchedule = do
  i <- ask
  pure $ case i `mod` 4 of
    0 -> (A0,B0)
    1 -> (A0,B1)
    2 -> (A1,B0)
    _ -> (A1,B1)
randomScheduleIO :: Scheduler IO
randomScheduleIO = do
  ra <- liftIO $ randomRIO (0 :: Int, 1)
  rb <- liftIO $ randomRIO (0 :: Int, 1)
  pure (if ra == 0 then A0 else A1, if rb == 0 then B0 else B1)
```

The fixed scheduler reads the step with `ask` and cycles through the four contexts. The random scheduler never reads the step: it flips two coins with `randomRIO`, lifted into the experiment monad by `liftIO`. We write $[\,a, b\,]_i$ for the probability that a scheduler picks (a, b) at step i. For `fixedSchedule` it is 1 or 0, and it changes with i; for `randomScheduleIO` it is $1/4$ for every context, at every step. The step is an index here, not an event: a subscript i on a probability always means "at step i".

The settings that a scheduler produces, step after step, are its *schedule*, and we speak of runs under the fixed and under the random schedule. Experiments in `IO` draw from the global generator of random numbers, and `seeded s` sets it to the generator of the seed `s` before it runs an experiment:

```haskell
seeded :: Int -> IO a -> IO a                                          L9
seeded s act = setStdGen (mkStdGen s) >> act
```

Every run in `IO` in this paper is wrapped in a `seeded s` of its own, with its own seed, so it starts from the same generator whatever ran before it; runs in `Identity` draw nothing at random. Figure 13 lists the seed of every run. Within a run, the draws are made step after step and, within a step, in the order of the code: first the scheduler's coins (under the random schedule two, Alice's then Bob's; under the fixed one none), then whatever the membrane and the boxes draw. The generator is the `StdGen` of Haskell's `random` library, version 1.3.1 on `splitmix` 0.1.2, which the build of this paper pins; another version may draw differently from the same seed.

3.3 One Experiment, One Table

We can now generalize the CHSH calculation of Section 2 to a given scheduler and any number of steps.

A trial. A trial runs the scheduler and the boxes at the same step, and keeps what an experimenter writes down: the two settings and the two outcomes.

```haskell
data Record = Record ASetting BSetting Outcome Outcome                 L10
  deriving Eq
trial :: (Monad e, TrialModel t)
      => Scheduler e -> (Context -> RunTrial e t) -> Exp e Record
trial sched membrane = do
  s@(a, b) <- sched
  (x, y)   <- membrane s (jointAB a b)
  pure (Record a b x y)
```

The scheduler and the membrane are both instructions of the experiment layer, so they read the same step i. The trial draws the settings once, names the pair `s`, and hands it to the membrane together with the trial computation `jointAB a b`. A trial thus takes a membrane for each context, of type `Context -> RunTrial e t`, which may depend on the settings it is handed. `blind runTrial` is one that does not: it ignores the settings and hands every trial computation to `runTrial`:

```haskell
blind :: RunTrial e t -> Context -> RunTrial e t                       L11
blind runTrial _ = runTrial
```

Every model in this paper but one uses such a membrane. The exception is the subject of Section 4.3.

A run. An experiment runs one trial at each of the steps $0, \ldots, n-1$ and returns the *record* of the run, one *page* per step:

```haskell
experiment :: Monad e => Int -> Exp e Record -> e [Record]             L12
experiment n oneTrial = mapM (runReaderT oneTrial) [0 .. n - 1]
```

The trial is an instruction of the experiment layer, so `runReaderT oneTrial` unwraps its function of the step, `Int -> e Record`, and `mapM` applies that function to each step in order. In the `IO` lab the coins are drawn in that order, so one seed fixes the whole record.

The table. The empirical model of a run is a table: a row for each context (a, b), a column for each pair of outcomes (x, y), and in each cell the number of trials that recorded that context and

those outcomes. A cell is thus a page that a trial can write, (a, b, x, y), and the table gives every page its count. The rows, the columns and the cells all come from the values of the settings and the outcomes. Their types derive `Enum` and `Bounded` (L1), so `every` lists the values of each, in the order of its declaration. The list applicative builds the rest: `liftA2 (,)`, which pairs the two outcomes in the default `jointAB`, here pairs every value of one list with every value of the other, and `<$>` with `<*>` fills the four fields of a page with every combination of values:

```
every :: (Bounded a, Enum a) => [a]                                    L13
every = [minBound .. maxBound]
contexts :: [Context]
contexts = liftA2 (,) every every
outcomePairs :: [(Outcome, Outcome)]
outcomePairs = liftA2 (,) every every
pages :: [Record]
pages = Record <$> every <*> every <*> every <*> every
```

The lists come out in the order of the tables in this paper, from `(A0, B0)` to `(A1, B1)` and from `(Plus, Plus)` to `(Minus, Minus)`. A table pairs each page with its count, and `empirical` tallies a record into one: `filter (== p)` keeps the pages of the record equal to `p`, and `length` counts them.

```
newtype Empirical = Empirical [(Record, Int)]                          L14
  deriving Eq
empirical :: [Record] -> Empirical
empirical rs = Empirical [ (p, length (filter (== p) rs)) | p <- pages ]
```

That is, $N(a, b, x, y) = \#\{i : \text{page}\, i = (a, b, x, y)\}$. With the all-plus boxes of Section 2, for instance, every page records the outcomes `(Plus, Plus)`, so every row has all its trials in its first column; divided by the number of trials of its context, it is the row of Figure 2. Tables derive `Eq`, so the accompanying code can compare two runs count for count.

Frequencies. These sixteen counts are all that a run reports; everything else is read off them. Divided by $N(a, b)$, the number of trials of its context, a count estimates the conditional probability of its pair of outcomes, $[\, a, b \vdash x, y \,] = N(a, b, x, y)/N(a, b)$. An `Event` picks some pairs of outcomes, such as those in which Bob answers `Plus`, or those in which the two outcomes agree. `numTrials` adds up the counts of the pairs it picks in context (a, b), and `freq` divides the sum by $N(a, b)$:

```
type Event = (Outcome, Outcome) -> Bool                                L15
numTrials :: Empirical -> ASetting -> BSetting -> Event -> Int
numTrials (Empirical m) a b p =
  sum [ k | (Record a' b' x y, k) <- m, (a', b') == (a, b), p (x, y) ]
freq :: Empirical -> ASetting -> BSetting -> Event -> Double
freq m a b p =
  fromIntegral (numTrials m a b p) / fromIntegral (numTrials m a b (const True))
```

So `freq m a b (== (x, y))` estimates $[\, a, b \vdash x, y \,]$, and an event that picks several pairs gets the sum of theirs, such as $[\, a, b \vdash x{=}{+} \,] = [\, a, b \vdash +, + \,] + [\, a, b \vdash +, - \,]$ for Alice's `Plus`.

The CHSH value. The CHSH value is read off the same table. A context now holds many trials, so it contributes their average score, the correlator $E(a, b)$ of Section 1: each pair of outcomes counts with its score xy, weighted by its frequency among the trials of that context:

$$E(a, b) = \sum_{x,y} xy\, [\, a, b \vdash x, y \,], \qquad \text{CHSH} = E(\texttt{A0}, \texttt{B0}) + E(\texttt{A0}, \texttt{B1}) + E(\texttt{A1}, \texttt{B0}) - E(\texttt{A1}, \texttt{B1}).$$

In the code, `correlator` is `trialValue` in expectation, and the CHSH value is `chshOf` of the four correlators, as `chshId` was `chshOf` of four scores:

```haskell
correlator :: Empirical -> ASetting -> BSetting -> Double
correlator m a b =
  sum [ fromIntegral (trialValue xy) * freq m a b (== xy) | xy <- outcomePairs ]
chsh :: (Monad e, TrialModel t)
     => Int -> Scheduler e -> RunTrial e t -> e Double
chsh n sched runTrial =
  chshOf . correlator . empirical <$> experiment n (trial sched (blind runTrial))
```
L16

Here `<$>` applies a function to the result of a computation: `chsh` runs the experiment with the settings that the scheduler provides, tallies its record, and computes the CHSH value from the table.[1] Deterministic boxes give every trial of a context the same outcomes, $\big(f(a), g(b)\big)$, so each correlator is the score $f(a)g(b)$, and the table gives CHSH(f, g) again. Figure 4 shows the whole pipeline.

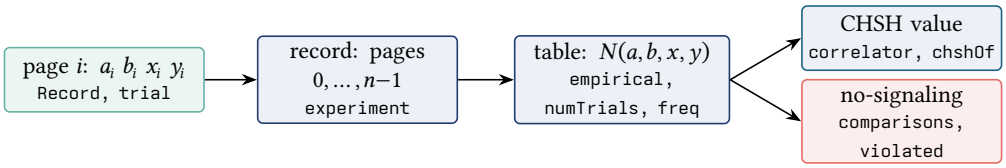


Figure 4: One run, one table. Each step writes a page; the record of the run is tallied into its empirical model, and both verdicts are read off that table.

3.4 Testing No-Signaling

The same table decides no-signaling. With outcomes ± 1, Equation (1) amounts to four equations, one per party and setting:

$$\underbrace{[\,a, \texttt{B0} \vdash x{=}{+}\,] = [\,a, \texttt{B1} \vdash x{=}{+}\,]}_{\text{Alice at } a} \qquad \underbrace{[\,\texttt{A0}, b \vdash y{=}{+}\,] = [\,\texttt{A1}, b \vdash y{=}{+}\,]}_{\text{Bob at } b} \tag{2}$$

Only two are displayed because each leaves its party's own setting free: Alice's stands for one equation at $a = \texttt{A0}$ and one at $a = \texttt{A1}$, and Bob's for one at each of $b = \texttt{B0}$ and $b = \texttt{B1}$. Each side is estimated from two counts of the run's table. A comparison names its condition, and holds for each of its two contexts the number of trials in which that party answered `Plus` and the number of all trials there:

```haskell
data Comparison = Comparison
  { condition :: String
  , first     :: (Int, Int)
  , second    :: (Int, Int)
  }
comparisons :: Empirical -> [Comparison]
comparisons m = [ alice a | a <- [A0, A1] ] ++ [ bob b | b <- [B0, B1] ]
  where
    alice a   = Comparison ("Alice at " ++ showA a) (plusA a B0) (plusA a B1)
    bob b     = Comparison ("Bob at " ++ showB b) (plusB A0 b) (plusB A1 b)
    plusA a b = (numTrials m a b (\(x, _) -> x == Plus), numTrials m a b anything)
```
L17

[1] Averaging over all trials the scores with the signs of their CHSH terms, times 4, gives the same value under the fixed schedule when 4 divides n, but not under a random one, where the contexts occur unequally often.

```
    plusB a b = (numTrials m a b (\(_, y) -> y == Plus), numTrials m a b anything)
    anything  = const True
```

The frequencies of a finite run are rarely exactly equal, so `violated` decides each equation with a two-proportion test. For frequencies $p_1 = k_1/n_1$ and $p_2 = k_2/n_2$,

$$p = \frac{k_1 + k_2}{n_1 + n_2}, \qquad se = \sqrt{p\,(1-p)\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}, \qquad \text{violated} \iff n_1 = 0 \vee n_2 = 0 \vee |p_1 - p_2| > 4\,se.$$

```
violated :: Comparison -> Bool                                                  L18
violated (Comparison _ (k1, n1) (k2, n2)) =
  n1 == 0 || n2 == 0 || abs (p1 - p2) > 4 * se
  where
    p1 = fromIntegral k1 / fromIntegral n1 :: Double
    p2 = fromIntegral k2 / fromIntegral n2
    p  = fromIntegral (k1 + k2) / fromIntegral (n1 + n2)
    se = sqrt (p * (1 - p) * (1 / fromIntegral n1 + 1 / fromIntegral n2))
```

Frequencies that agree exactly are never flagged, and frequencies 1 and 0 are flagged as soon as the two contexts together have more than 16 trials. At four standard errors, a large run of a model that does not signal is flagged in fewer than one run in three thousand.

A report holds the table of a run, its CHSH value, and the conditions that the run violates; we say that a run *signals* when it violates at least one. `report` runs the experiment once, builds its table, and reads both verdicts off it; `chshReport` takes the same arguments as `chsh`:

```
data Report = Report                                                            L19
  { table     :: Empirical
  , chshValue :: Double
  , signaling :: [String]
  }
report :: Monad e => Int -> Exp e Record -> e Report
report n oneTrial = do
  m <- empirical <$> experiment n oneTrial
  pure (Report m (chshOf (correlator m))
                 [ condition c | c <- comparisons m, violated c ])
chshReport :: (Monad e, TrialModel t)
           => Int -> Scheduler e -> RunTrial e t -> e Report
chshReport n sched runTrial = report n (trial sched (blind runTrial))
noSignalingIn :: Report -> Bool
noSignalingIn = null . signaling
```

Every CHSH value and every verdict in the rest of this paper comes from such a report: one run of 80000 trials, one table, both verdicts. We run each model under the fixed schedule, the random one, or both; under the fixed schedule, each context gets 20000 trials. Figure 5 collects the vocabulary.

The two models of Section 2 already show the check at work. Pure boxes give CHSH 2.000 and do not signal.[2] The boxes with a shared location give CHSH 4.000, and Bob's frequency of `Plus` at `B1` is 0.000 next to `A0` and 1.000 next to `A1`: he signals, as intended.

[2]This run uses the pure boxes of the accompanying module `CHSH.Identity`: Alice answers `Plus` at `A0` and `Minus` at `A1`, and Bob the reverse. The all-plus boxes are a second instance for `Identity`, in a module of its own.

Term	Meaning	In the code
setting	a party's input to its box	`ASetting`, `BSetting`
context	the two settings (a, b) of a trial	`Context`
outcome	what a box answers: $+1$ or -1	`Outcome`
trial	one use of both boxes, in one context	`jointAB`, `trial`
step	the position i of a trial in an experiment	the `Int` of `Exp e`
scheduler	chooses the settings of each step: its *schedule*	`Scheduler e`
membrane	runs a trial computation for the experiment	`RunTrial e t`
run	one experiment: n trials, at the steps $0, \ldots, n-1$	`experiment`
record	what a run writes down: a *page* (a, b, x, y) per trial	`[Record]`
table	the counts $N(a, b, x, y)$ of a record: its empirical model	`Empirical`
verdicts	the CHSH value of a table, and whether it *signals*	`Report`

Figure 5: The vocabulary of an experiment.

3.5 Scheduler-Induced Loopholes

The two models so far cannot tell one schedule from another, because their outcomes depend on the settings alone. Here are *clocked* boxes, which also consult the current step of the experiment:

```haskell
newtype Clocked a = Clocked { unClocked :: Reader Int a }                    L20
  deriving (Functor, Applicative, Monad, MonadReader Int)
instance TrialModel Clocked where
  localA _ = pure Plus
  localB _ = do
    i <- ask
    pure $ if i `mod` 4 == 3 then Minus else Plus
```

Alice always answers `Plus`, and Bob answers `Minus` on every fourth step. Two membranes run these boxes. One passes the current step on to them; the other hides it, and runs every trial as if at step 0:

```haskell
runTrialClockLoose :: Applicative e => RunTrial e Clocked                    L21
runTrialClockLoose (Clocked r) = ReaderT (\i -> pure (runReader r i))
runTrialClockTight :: Applicative e => RunTrial e Clocked
runTrialClockTight (Clocked r) = ReaderT (\_ -> pure (runReader r 0))
```

Two membranes under two schedules make four runs. Their CHSH values concentrate around 1, 2, 2, and 4, in some order. We invite the reader to pause, to match each value with its run, and to guess which of the runs signal.

❧ ❧

Under the fixed schedule, the settings follow a known pattern: `(A0, B0)`, `(A0, B1)`, `(A1, B0)`, and finally `(A1, B1)`. Bob follows a pattern too: he disagrees with Alice exactly when the step satisfies $i \bmod 4 = 3$. When the membrane shows the step to the boxes, the two patterns coincide. Every fourth step measures `(A1, B1)`, the one context in which the CHSH expression rewards disagreement. All four terms then contribute $+1$, and the run reaches the algebraic maximum 4. Its report, as `printReport` prints it, shows how:

```
a   b   |  ++     +-     -+     --    | P(x=+) P(y=+)  trials
A0  B0  | 1.000  0.000  0.000  0.000  | 1.000  1.000  20000
A0  B1  | 1.000  0.000  0.000  0.000  | 1.000  1.000  20000
A1  B0  | 1.000  0.000  0.000  0.000  | 1.000  1.000  20000
A1  B1  | 0.000  1.000  0.000  0.000  | 1.000  0.000  20000
CHSH = 4.0
Alice at A0  1.000  vs  1.000   ok
Alice at A1  1.000  vs  1.000   ok
Bob at B0    1.000  vs  1.000   ok
Bob at B1    1.000  vs  0.000   violated
× No-signaling VIOLATED in this run: Bob at B1
```

Bob answers `Minus` exactly in context `(A1, B1)`, so his frequency of `Plus` at `B1` follows Alice's setting: 1.000 next to `A0`, 0.000 next to `A1`.

This coordination involves no communication during a trial. It comes from a correlation between the schedule of the experiment and the strategy of the boxes: boxes that can read the experiment's clock can anticipate its settings. The boxes do not signal to each other: the schedule signals to Bob, through the step.

The random schedule confirms this explanation. Bob still answers `Minus` on every fourth step, but these steps now fall on all four contexts alike, so in each context the two outcomes agree on three trials in four. Every correlator is then $3/4 - 1/4 = 1/2$, and the CHSH value collapses to $1/2 + 1/2 + 1/2 - 1/2 = 1$ (0.990 in our run); the table shows no signaling. Hiding the step from the boxes restores purely local behavior: the classical value 2, without signaling, under either schedule.

The lesson is not that stronger effects yield larger CHSH values. It is that violations arise when the strategy of the boxes becomes correlated with the experimental protocol. A fixed, predictable schedule invites such a correlation; a random one destroys it. A model of a Bell experiment must therefore control not only the effects inside the boxes, but also how information about the experiment as a whole reaches them. The same boxes also show that no-signaling is a property of a run, not of the boxes alone: they signal under one schedule and not under the other. A check that ran them apart from the run's schedule could not tell the two runs apart (Section 8).

4 The λ-Model in Code: Blind, Local, Factored

The clocked boxes reached the algebraic maximum of 4 without any communication during a trial. They did not refute Bell's theorem. They exposed a weakness of the experimental protocol: a schedule that the boxes could follow.

Bell's own analysis avoids such pitfalls by giving all variation from trial to trial a name: a hidden variable, traditionally written λ (Bell, 1964). It stands for whatever the boxes may share before a trial, such as an agreed strategy or shared randomness, and it is assumed to be independent of the settings chosen in that trial. Figure 6 shows this λ-model.

In our framework, λ is not a new concept but a name for what state, clocks and other effects have been doing all along: varying the behavior of the boxes across trials. Bell's theorem rests on three assumptions about how they may do so. We state each as a property of a model, named by what it asks of the program, with the name of the assumption in the literature on Bell's theorem in parentheses (Section 10):

- **Blind** (λ-independence). The strategy is independent of the settings, $[\, a, b \vdash \lambda \,] = [\, \lambda \,]$: the code that draws λ does not see them.

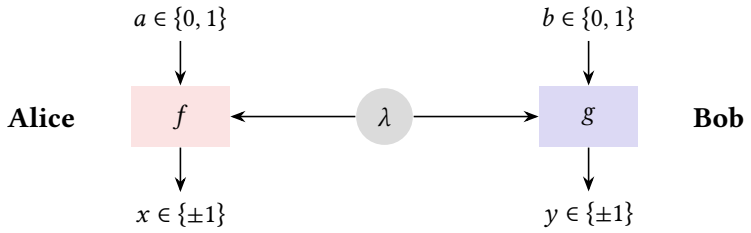


Figure 6: The λ-model: two boxes f and g with a hidden variable λ.

- **Local** (parameter independence). Each box answers from its own setting and λ alone, whatever the other setting: $[\, a, b, \lambda \vdash x \,] = [\, a, \lambda \vdash x \,]$ and $[\, a, b, \lambda \vdash y \,] = [\, b, \lambda \vdash y \,]$.
- **Factored** (outcome independence). Given the settings and λ, the two outcomes are independent: $[\, a, b, \lambda \vdash x, y \,] = [\, a, b, \lambda \vdash x \,] \cdot [\, a, b, \lambda \vdash y \,]$.

Together they bound the CHSH value by 2 (Bell, 1964; Brandenburger and Yanofsky, 2008). For a fixed λ, local and factored boxes answer, each from its own setting, independently of each other, so the regrouping of Section 1 applies to their average outcomes, which lie between -1 and $+1$, and gives the same bound. Blindness lets us average the bound over λ, because λ has the same distribution in every context.

Each form of non-classical behavior in this paper comes from giving up one of the three properties. The loopholes of Sections 3.5 and 4.2 and the superdeterminism of Section 4.3 are not blind. The quantum boxes of Section 5 and the PR boxes of Section 6 are not factored. The shared location of Section 2 and the hidden states of Section 7 are not local. This section first makes the three properties explicit in code: local and factored in the boxes (Section 4.1), blind in the sampler that draws λ (Section 4.2), whose type alone, we will see, does not make it so. It then gives up blindness outright (Section 4.3).

4.1 Local Hidden Variables as Immutable Data

In the CHSH scenario, a deterministic local strategy says in advance which outcome each box gives at each of its settings. It consists of four outcomes:

```
data Lambda = Lambda                                                    L22
  { a0 :: Outcome
  , a1 :: Outcome
  , b0 :: Outcome
  , b1 :: Outcome
  } deriving Eq
```

A value of type `Lambda` describes the complete behavior of both boxes in one trial. For example,

```
lambdaAllPlus :: Lambda                                                 L23
lambdaAllPlus = Lambda Plus Plus Plus Plus
```

always produces equal outcomes and yields a CHSH value of $+2$, while

```
lambdaAntiAll :: Lambda                                                 L24
lambdaAntiAll = Lambda Plus Plus Minus Minus
```

always produces opposite outcomes and yields a CHSH value of -2. There are sixteen such strategies, and the list `allLambdas` enumerates them:

```haskell
allLambdas :: [Lambda]
allLambdas =
  [ Lambda x0 x1 y0 y1
  | x0 <- allOutcomes
  , x1 <- allOutcomes
  , y0 <- allOutcomes
  , y1 <- allOutcomes
  ]
  where allOutcomes = [Minus, Plus]
```
L25

The last generator varies fastest, and each runs from `Minus` to `Plus`, the reverse of the order of `every` (L13). So the entries `a0`, `a1`, `b0`, `b1` of the strategy at index j are the binary digits of j, most significant first, with `Minus` for 0 and `Plus` for 1: `b1` is `Plus` when j is odd, and `b0` is `Plus` when j mod 4 is 2 or 3. This order will matter for a sampler that goes through the list with the step (Section 4.2).

The boxes read their strategy from a `Reader` monad (`LHV` stands for *local hidden variables*); `asks a0` reads the field `a0` of λ:

```haskell
newtype LHV a = LHV { unLHV :: Reader Lambda a }
instance TrialModel LHV where
  localA A0 = asks a0
  localA A1 = asks a1
  localB B0 = asks b0
  localB B1 = asks b1
```
L26

Note that λ is a *deterministic* strategy; randomness enters only through how λ is sampled across trials.

Local and factored. These boxes have both properties by construction. They are local: `localA` receives an `ASetting` and has access to λ, but none to a `BSetting`, and likewise `localB`; and a reader monad lets both boxes read λ, but lets neither write anything that the other could read. They are factored: `jointAB` keeps its default (L2), which runs the two reads and pairs their results. Locality needs both, the types and the effect. The shared location of Section 2 has methods of the same types and is not local: `State` lets Alice write her setting where Bob reads it.

4.2 Blind Sampling

It remains to say how λ varies across trials, and what keeps it blind to the settings.

Sampling λ. In our model, λ is supplied by a *sampler*, an instruction of the experiment layer, like the scheduler:

```haskell
type Sampler e = Exp e Lambda
```
L27

A sampler may read the step, so the hidden variable can change from trial to trial, but it is not handed the settings. Here are two. `pure` makes an instruction that ignores the step, and `ReaderT`, as before, makes one from a function of the step:

```haskell
plus2, uniformAll :: Applicative e => Sampler e
plus2      = pure lambdaAllPlus
uniformAll = ReaderT (uniformCycle allLambdas)
```
L28

where `uniformCycle xs i` picks `xs !! (i `mod` length xs)`. So `plus2` always draws the strategy `lambdaAllPlus`, and `uniformAll` goes through all sixteen strategies in turn. The accompanying code has a few more: `minus2` always draws `lambdaAntiAll`, and `altExtrema` alternates between the two from step to step. The membrane wires the sampler into the trial; `runReader r` runs the boxes' reader computation with the strategy that `samp` draws:

```
runTrialLHV :: Functor e => Sampler e -> RunTrial e LHV                    L29
runTrialLHV samp (LHV r) = runReader r <$> samp
```

The hidden variable is thus drawn outside the trial, by an instruction of the experiment layer, and supplied to the boxes as an immutable environment. Whatever happens inside the trial can observe the λ that was chosen, and cannot influence how it was chosen.

Note that the membrane draws λ *after* the trial's settings (L10). What keeps λ independent of the settings is not the order of the two draws but the flow of data: the membrane ignores the settings, so their values cannot flow into λ. In terms of the experiment monad, the settings and the strategy of one step are composed applicatively:

```
independent :: Monad e => Scheduler e -> Sampler e -> Exp e (Context, Lambda)   L30
independent sched samp = liftA2 (,) sched samp
```

`liftA2` runs `sched` and then `samp`, but `samp` cannot see the value that `sched` returns. (The accompanying code checks that feeding this pair to the boxes gives the record of `trial` with the membrane `blind (runTrialLHV samp)`.) We can now run several instructive samplers.

Extremal strategies. Fixing λ to one of the extremal strategies, with `plus2` or `minus2`, yields the classical bounds exactly: CHSH 2.000 and -2.000, with no signaling. No model that is blind, local and factored can exceed these values.

Uniform strategies. If we cycle through all sixteen strategies with `uniformAll`, positive and negative correlations cancel each other. Under the random schedule, the run's CHSH value is close to 0 (0.008), and the run does not signal. Its table is the uniform empirical model, up to sampling fluctuations: in each context, every pair of outcomes occurs with frequency about $1/4$.

a	b	$(+,+)$	$(+,-)$	$(-,+)$	$(-,-)$	$[\,x{=}+\,]$	$[\,y{=}+\,]$	trials
A0	B0	0.254	0.247	0.254	0.245	0.501	0.508	19868
A0	B1	0.246	0.253	0.251	0.250	0.499	0.497	20018
A1	B0	0.251	0.249	0.247	0.253	0.500	0.498	20040
A1	B1	0.245	0.252	0.253	0.251	0.496	0.497	20074

Under the fixed schedule the CHSH value is exactly 0, yet the run signals, and its table is far from uniform:

a	b	$(+,+)$	$(+,-)$	$(-,+)$	$(-,-)$	$[\,x{=}+\,]$	$[\,y{=}+\,]$	trials
A0	B0	0.000	0.500	0.000	0.500	0.500	**0.000**	20000
A0	B1	0.500	0.000	0.500	0.000	0.500	1.000	20000
A1	B0	0.500	0.000	0.500	0.000	0.500	**1.000**	20000
A1	B1	0.500	0.000	0.500	0.000	0.500	1.000	20000

The sampler cycles with the step, and so does the scheduler, so i mod 4 fixes the context and Bob's entries of λ at once. In the order of `allLambdas` (L25), the steps with i mod $4 = 0$ measure `(A0, B0)`

and draw a strategy whose `b0` is `Minus`, those with $i \bmod 4 = 2$ measure `(A1, B0)` and draw one whose `b0` is `Plus`, and at the other two steps of every four, which measure `(A0, B1)` and `(A1, B1)`, `b1` is `Plus`. So in context `(A0, B0)` Bob always answers `Minus`, and in `(A1, B0)` always `Plus` (the marked frequencies). The sampler is never handed the settings, and yet, through the step, its λ is not independent of them. The CHSH value does not show this dependence; the table does.

Exploiting the step. A sampler that reads the step can do worse:

```haskell
biasPlus2Loophole, biasPlus2Blocks :: Applicative e =>                    L31
  Int -> Int -> Sampler e
biasPlus2Loophole = \k m -> ReaderT $
  \i -> if (i `mod` m) < k then pure lambdaAllPlus else pure lambdaAntiAll
biasPlus2Blocks = \k m -> ReaderT $
  \i -> let blk = i `div` 4
        in pure $ if (blk `mod` m) < k then lambdaAllPlus else lambdaAntiAll
```

`biasPlus2Loophole 7 8` draws the all-plus strategy unless $i \bmod 8 = 7$, and the anti-correlated one when it is. Under the fixed schedule, a step with $i \bmod 8 = 7$ always measures `(A1, B1)`, the context of the minus term in CHSH. The run reaches CHSH 3.000, beyond the classical bound, with a sampler of type `Sampler e` and local deterministic boxes; its table shows Bob at `B1` signaling (1.000 next to `A0`, 0.500 next to `A1`). Its variant `biasPlus2Blocks 7 8` draws one strategy per block of four steps and so cannot single out a context: it gives $4 \cdot \frac{7}{8} - 2 = 1.500$ and does not signal.

Why the type is not enough. So the type of the sampler does not make a model blind. It keeps the settings themselves out of the sampler's hands, but not what determines them: the scheduler and the sampler read the same step, and through it a sampler can follow a scheduler that reads the step. In terms of probabilities, let $[\,\lambda\,]_i$ be the probability that the sampler draws λ at step i, and $[\,a,b \vdash \lambda\,]_i$ that of drawing it there when the settings are (a,b). The run's table pools the steps of each context: in context (a,b) it sees λ with probability

$$\frac{\sum_i [\,a,b\,]_i\,[\,a,b \vdash \lambda\,]_i}{\sum_i [\,a,b\,]_i}. \tag{3}$$

A membrane that ignores the settings gives $[\,a,b \vdash \lambda\,]_i = [\,\lambda\,]_i$ at every step, as long as the lab's effect `e` holds nothing that the scheduler wrote. The pooled distribution (3) is then the same in every context when $[\,\lambda\,]_i$ does not change with i, or when $[\,a,b\,]_i$ does not, as under the random schedule. The cycling sampler under the fixed schedule satisfies neither condition. A sampler that cannot read the step satisfies the first, provided that its action draws in the same way at every call. `fresh` makes such a sampler from an action of the lab alone, and `uniformIO` is an action that draws one of the sixteen strategies from coins:

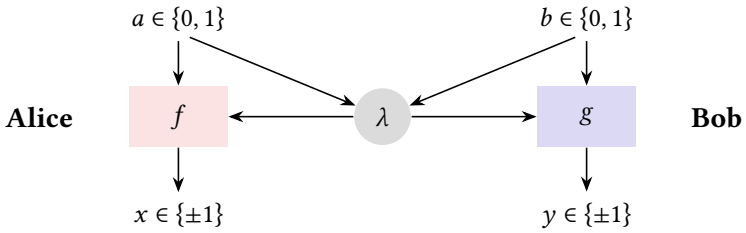
```haskell
fresh :: e Lambda -> Sampler e                                            L32
fresh draw = ReaderT (const draw)
uniformIO :: IO Lambda
uniformIO = (allLambdas !!) <$> randomRIO (0, length allLambdas - 1)
```

Under the fixed schedule, `fresh uniformIO` gives CHSH 0.024 and does not signal, where the cycling `uniformAll` signaled.

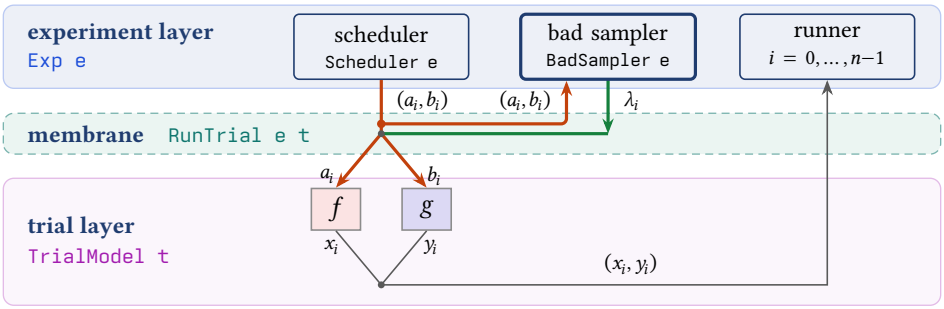
All of these samplers followed the settings at most indirectly, through the step. Section 4.3 hands the settings to the sampler outright.

4.3 Superdeterminism: A Sampler That Sees the Settings

The samplers of Section 4.2 are not handed the settings. Superdeterministic models give up exactly this: informally, superdeterminism (Hossenfelder and Palmer, 2020; Palmer, 2009; 't Hooft, 2016) allows the hidden variable λ to depend on the measurement settings themselves, which some presentations describe as a failure of "free choice" (Conway and Kochen, 2006, 2009). In our framework, it means letting the code that draws λ see the settings (a, b) (Figure 7). Nothing non-classical needs to happen inside the boxes: they may remain local, factored and deterministic.



(a) The superdeterministic λ-model.



(b) In the architecture: the trial hands the settings of the step to the membrane (L10), and the membrane passes them on to a bad sampler (L34), whose strategy λ_i comes back down to the boxes.

Figure 7: Superdeterminism.

A sampler that sees the settings. Such a sampler maps the settings to an instruction of the experiment layer. We call it a *bad sampler*, and a sampler of type `Sampler e` a *legal* one:

```
type BadSampler e = Context -> Exp e Lambda                                L33
```

A legal sampler is a bad one that ignores its argument, `const samp`. The trial already hands the settings it drew to the membrane (L10); the superdeterministic membrane passes them on to the bad sampler, and is otherwise that of local hidden variables:

```
runTrialSuperdet :: Functor e => BadSampler e -> Context -> RunTrial e LHV   L34
runTrialSuperdet bad s = runTrialLHV (bad s)
```

A superdeterministic run thus goes through the same `trial` as every other model, with the membrane `runTrialSuperdet bad` where the others use `blind runTrial`. In terms of the experiment monad, the applicative pair of L30 becomes a bind, which passes the settings that `sched` returns on to `bad`:

```
dependent :: Monad e => Scheduler e -> BadSampler e -> Exp e (Context, Lambda)   L35
dependent sched bad = sched >>= \s -> (,) s <$> bad s
```

With `bad = const samp` it is the draw of L30 again. (The accompanying code checks that this pair, fed to the boxes, gives the record of `trial sched (runTrialSuperdet bad)`.)

A maximally cheating sampler. With the settings in hand, it is easy to make every CHSH term contribute +1. In context `(A1, B1)` the sampler draws a strategy on which Alice and Bob disagree, so that the minus sign turns the term positive; in every other context, one on which they agree:

```haskell
cheatLam :: Applicative e => BadSampler e                                  L36
cheatLam (a,b) =
  pure $ case (a,b) of
    (A1,B1) -> Lambda Plus Plus Plus Minus
    _       -> Lambda Plus Plus Plus Plus
```

Its runs reach the algebraic maximum with local, factored, deterministic boxes: CHSH 4.000 under the fixed schedule, and 4.000 under the random one. Unlike for the loopholes of Sections 3.5 and 4.2, randomizing the settings does not help. This is the defining feature of superdeterminism, and in code the correlation is explicit: the sampler is handed the settings. A bad sampler that ignores them, `legalAllPlus _ = plus2`, gives 2.000.

The table sees it, or not. Both runs of `cheatLam` signal: at `B1`, Bob answers `Plus` next to `A0` and `Minus` next to `A1` (frequencies 1.000 and 0.000 in the random run). Local boxes cannot signal by themselves; it is the dependence of λ on the settings that moves Bob's marginal, and the run's table records the move. The dependence need not move a marginal, though:

```haskell
cheatNS :: BadSampler IO                                                   L37
cheatNS (a, b) = liftIO $ do
  r <- randomOutcome
  let y = if a == A1 && b == B1 then flipOutcome r else r
  pure (Lambda r r y y)
```

`cheatNS` flips one fair coin with `randomOutcome`[3] and hands it to both boxes, turned over by `flipOutcome` for Bob in context `(A1, B1)`. Under the random schedule, its run reproduces the PR box of Section 6, CHSH value 4 included, and it does not signal:

```
a   b   |  ++     +-     -+     --    | P(x=+) P(y=+)  trials
A0  B0  | 0.498  0.000  0.000  0.502  | 0.498  0.498  19845
A0  B1  | 0.495  0.000  0.000  0.505  | 0.495  0.495  20447
A1  B0  | 0.500  0.000  0.000  0.500  | 0.500  0.500  19939
A1  B1  | 0.000  0.500  0.500  0.000  | 0.500  0.500  19769
CHSH = 4.0
Alice at A0  0.498  vs  0.495   ok
Alice at A1  0.500  vs  0.500   ok
Bob at B0    0.498  vs  0.500   ok
Bob at B1    0.495  vs  0.500   ok
✓ No-signaling holds in this run
```

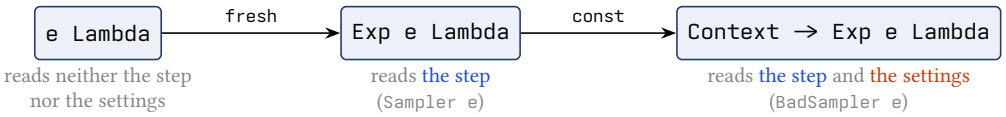
No test on the table can detect this dependence: local deterministic boxes with a sampler that sees the settings are, as far as the record of a run can tell, a PR box.

[3] `randomOutcome` draws an `Int` between 0 and 1 with `randomRIO`, and answers `Minus` for 0 and `Plus` for 1; `flipOutcome` swaps the two outcomes.

A sampler that peeks. The bind is open to a sampler of the legal type, too, because the sampler and the scheduler read the same step:

```haskell
peek :: Monad e => Scheduler e -> BadSampler e -> Sampler e
peek sched bad = sched >>= bad
```
L38

`peek sched bad` runs the scheduler itself and hands the settings it gets to a bad sampler; nothing in the type of the sampler that it returns says so. Under `fixedSchedule`, a function of the step, it finds the trial's own settings, and `runTrialLHV (peek fixedSchedule cheatLam)` reproduces the run of `cheatLam`: CHSH 4.000, with Bob at `B1` signaling. Under `randomScheduleIO` it flips coins of its own, and the cheat is gone: CHSH 1.990, with no signaling. The three types of sampler thus embed into one another, each reading more than the last:



Only the first rules out a dependence on the settings by itself, and only if the lab's effect `e` carries nothing to it from the scheduler or from earlier steps; the second rules it out only together with a scheduler that does not read the step.

In executable terms, superdeterminism is the refactoring that turns the applicative pair of scheduler and sampler into a bind. It needs no signaling and no joint effects, which is why Bell's theorem must exclude it by assumption: without blindness, the CHSH inequality ceases to distinguish classical from non-classical correlations. In the next two sections the models stay blind and local, and give up only factoring.

5 Quantum Correlations in Code

So far, every model has left one method of the interface untouched. Every `TrialModel` provides not only the local responses `localA` and `localB`, but also the joint response `jointAB`, and we have relied on its default (L2), which runs Alice's and Bob's local computations and pairs their results. With local hidden variables, that default is what made the boxes factored.

Quantum mechanics departs from classical behavior here. It does not let one box signal to the other, and it does not need a hidden variable that depends on the measurement settings. Each box, on its own, answers as its local response says; but once Alice's box has answered, Bob's box no longer answers as it would on its own. In our framework, this departure is expressed entirely by overriding `jointAB`.

What this model is (and is not). We write the quantum boxes as a quantum computing textbook writes them (Nielsen and Chuang, 2010): the state of two qubits is a vector of amplitudes, a measurement is a pair of projectors, and the Born rule gives the probability of each outcome and the state that the outcome leaves. Real amplitudes suffice for the CHSH experiment, so the linear algebra is that of lists of real numbers: vectors of four entries, and matrices of two by two or four by four. The model does not simulate a physical apparatus, and it is not a general simulator of quantum programs, as the Quantum IO monad of Altenkirch and Green (2010) is: it computes only the statistics of measuring two qubits. Sections 5.1 and 5.2 set up this linear algebra; a reader who knows it can go straight to Section 5.3.

5.1 Qubits and Measurements

A qubit has two basis states, $|0\rangle$ and $|1\rangle$, and a pair of qubits has four, $|00\rangle$, $|01\rangle$, $|10\rangle$ and $|11\rangle$: the first digit for Alice's qubit, the second for Bob's. A state of the pair is a vector of four *amplitudes*, one for each basis state; their squares are the probabilities of finding the pair in each basis state, and add up to 1. An operator on the pair is a four-by-four matrix. Both are lists of numbers here.

We need three operations of linear algebra, and we write them as Dirac does: the inner product $\langle u|v\rangle$ of two vectors, a matrix applied to a vector, $M|v\rangle$, and the tensor (Kronecker) product $M \otimes N$, which makes an operator on the pair from one on each qubit. Haskell has no mixfix operators, but two infix ones come close: `u <| v` is $\langle u|v\rangle$ and `m |> v` is $M|v\rangle$, and since `|>` binds tighter than `<|`, the sandwich $\langle\psi|M|\psi\rangle$ reads `psi <| m |> psi`. The tensor product, spelled `><` in the source, is printed $\otimes$ here:

```haskell
type Vector = [Double]                                                  L39
type Matrix = [[Double]]
infixr 6 <|
infixr 7 |>
infixl 8 ⊗
(<|) :: Vector -> Vector -> Double
u <| v = sum (zipWith (*) u v)
(|>) :: Matrix -> Vector -> Vector
m |> v = map (<| v) m
(⊗) :: Matrix -> Matrix -> Matrix
m ⊗ n = [ concat [ map (a *) row | a ← mrow ] | mrow ← m, row ← n ]
```

The module also defines `add`, `sub` and `scale`, the entrywise sum and difference of two matrices and a multiple of one, and `normalize`, which divides a vector v by its length $\sqrt{\langle v|v\rangle}$.

Each setting measures an *observable*: a two-by-two matrix whose eigenvalues, $+1$ and -1, are the outcomes `Plus` and `Minus`. Alice measures one of the Pauli matrices Z and X, and Bob one of their normalized sum and difference:

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \qquad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \frac{Z+X}{\sqrt{2}}, \qquad \frac{Z-X}{\sqrt{2}}.$$

These give the largest CHSH value for the maximally entangled state below; the textbook's account of the CHSH inequality measures the same observables, up to sign, on another entangled state (Nielsen and Chuang, 2010, Section 2.6):

```haskell
identity, pauliZ, pauliX :: Matrix                                      L40
identity = [[1, 0], [0, 1]]
pauliZ   = [[1, 0], [0, -1]]
pauliX   = [[0, 1], [1, 0]]
observableA :: ASetting -> Matrix
observableA A0 = pauliZ
observableA A1 = pauliX
observableB :: BSetting -> Matrix
observableB B0 = scale (1 / sqrt 2) (add pauliZ pauliX)
observableB B1 = scale (1 / sqrt 2) (sub pauliZ pauliX)
```

Each outcome of an observable O has a *projector*, the matrix that keeps the part of a state in which O has that outcome: P_+ for `Plus` and P_- for `Minus`. On the pair, Alice's projectors act on her qubit and Bob's on his:

$$P_\pm = \frac{I \pm O}{2}, \qquad P \otimes I \text{ for Alice}, \qquad I \otimes P \text{ for Bob}.$$

`measA` and `measB` build these projectors on the pair:

```
projector :: Matrix -> Outcome -> Matrix                                    L41
projector o Plus  = scale (1 / 2) (add identity o)
projector o Minus = scale (1 / 2) (sub identity o)
measA :: ASetting -> Outcome -> Matrix
measA a x = projector (observableA a) x ⊗ identity
measB :: BSetting -> Outcome -> Matrix
measB b y = identity ⊗ projector (observableB b) y
```

The Born rule gives the probability of the outcome with projector P in the state $|\psi\rangle$, and, when that probability is positive, the state that the outcome leaves:

$$\langle\psi|P|\psi\rangle, \qquad \frac{P|\psi\rangle}{\sqrt{\langle\psi|P|\psi\rangle}}.$$

In code, the projector `p` plays P: `probability p psi` is `psi <| p |> psi`, and `collapse` divides $P|\psi\rangle$ by its length, which is the square root of that probability. `coin p` answers `Plus` with probability `p`:

```
probability :: Matrix -> Vector -> Double                                   L42
probability p psi = psi <| p |> psi
collapse :: Matrix -> Vector -> Vector
collapse p psi = normalize (p |> psi)
coin :: Double -> IO Outcome
coin p = do
  u <- randomRIO (0, 1)
  pure (if u < p then Plus else Minus)
```

5.2 An Entangled Pair

A source prepares the pair in a state. The maximally entangled state $|\Phi^+\rangle$ gives equal amplitudes to $|00\rangle$ and $|11\rangle$, and none to the other two: the qubits are both 0 or both 1. A parameter θ weighs the two terms, and the states $|\psi_\theta\rangle$ include $|\Phi^+\rangle$ at $\theta = \pi/4$:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2}}[1,0,0,1], \qquad |\psi_\theta\rangle = \cos\theta\,|00\rangle + \sin\theta\,|11\rangle.$$

At $\theta = 0$ the pair is in $|00\rangle$, which is not entangled at all:

```
type Source = Vector                                                        L43
maximallyEntangled :: Source
maximallyEntangled = normalize [1, 0, 0, 1]
entangled :: Double -> Source
entangled theta = [cos theta, 0, 0, sin theta]
```

Either qubit of $|\Phi^+\rangle$, on its own, gives each outcome with probability $1/2$, whatever is measured. When Alice measures her qubit, the state collapses, and Bob's qubit is left in the state in which her observable is certain to give her outcome. Measuring Z leaves $|00\rangle$ for `Plus` and $|11\rangle$ for `Minus`; measuring X leaves $|{+}{+}\rangle$ for `Plus` and $|{-}{-}\rangle$ for `Minus`. Here $|\pm\rangle = (|0\rangle \pm |1\rangle)/\sqrt{2}$ are the states of one qubit in which X is certain to give `Plus` or `Minus`, and $|00\rangle + |11\rangle$ is also $|{+}{+}\rangle + |{-}{-}\rangle$.

The correlator of a context, the average of xy, is

$$E(a,b) = \langle\psi|A \otimes B|\psi\rangle$$

for Alice's observable A and Bob's observable B, and `chshOf (correlation psi)` is the exact CHSH value of a state:

```
correlation :: Source → ASetting → BSetting → Double                    L44
correlation psi a b = psi <| observableA a ⊗ observableB b |> psi
```

For $|\Phi^+\rangle$, the averages $\langle\psi|Z \otimes Z|\psi\rangle$ and $\langle\psi|X \otimes X|\psi\rangle$ are 1, and those of $Z \otimes X$ and $X \otimes Z$ are 0. So the correlator is $1/\sqrt{2}$ in the three contexts other than `(A1, B1)`, and $-1/\sqrt{2}$ in that one. The CHSH value is $4/\sqrt{2} = 2\sqrt{2}$, the Tsirelson bound (Tsirelson, 1980). For $|\psi_\theta\rangle$, the average of $X \otimes X$ is $\sin 2\theta$ and the others stay as they are, so

$$\mathrm{CHSH}(\theta) = \sqrt{2}\,(1 + \sin 2\theta).$$

The parameter moves the CHSH value from $\sqrt{2} \approx 1.414$ at $\theta = 0$ up to the bound $2\sqrt{2}$ at $\theta = \pi/4$. On the way it exceeds the classical bound 2 once $\sin 2\theta > \sqrt{2} - 1$, that is, from $\theta \approx 0.214$ on, about 12.2°. These observables are the best ones only at $\theta = \pi/4$; for any other θ, other observables give a larger value.

5.3 Overriding `jointAB`

The trial monad holds the state of the pair, and flips coins in `IO`. The local responses read the state with `get`, measure each qubit on its own, and leave the state as it is. The joint response, however, pairs two computations that also write to the state:

```
newtype Quantum a = Quantum { unQuantum :: StateT Vector IO a }          L45
  deriving newtype (Functor, Applicative, Monad, MonadIO, MonadState Vector)
instance TrialModel Quantum where
  localA a = do
    psi ← get
    liftIO (coin (probability (measA a Plus) psi))
  localB b = do
    psi ← get
    liftIO (coin (probability (measB b Plus) psi))
  jointAB a b = liftA2 (,) (nonlocalA a) (nonlocalB b)
```

Alice's, `nonlocalA`, answers as her local response does, and then collapses the state to what her outcome leaves. Bob's, `nonlocalB`, is its dual:

```
nonlocalA :: ASetting → Quantum Outcome                                  L46
nonlocalA a = do
  x ← localA a
  modify (collapse (measA a x))
  pure x
nonlocalB :: BSetting → Quantum Outcome
nonlocalB b = do
  y ← localB b
  modify (collapse (measB b y))
  pure y
```

The default pairs the local responses with `liftA2 (,)`, and the override pairs these two computations in the same way. In a monad, `liftA2` runs the first computation and then the second, so Bob's local response reads the state that Alice's outcome left, and his outcome depends on hers. The pair stays applicative: `nonlocalB` never sees the value of her outcome, and the dependence travels

through the state. In this order nothing reads the state after Bob's write, but either box could go first, as we see below.

The membrane supplies the state, as the membrane of local hidden variables supplies a strategy: `evalStateT m psi` runs a trial from the state `psi`, and drops the state in which the trial ends. The membrane ignores the step, so no information about the schedule can reach the boxes:

```haskell
runTrialSource :: Source -> RunTrial IO Quantum                                  L47
runTrialSource psi (Quantum m) = ReaderT (\_ -> evalStateT m psi)
runTrialQuantum :: RunTrial IO Quantum
runTrialQuantum = runTrialSource maximallyEntangled
```

Local, but not factored. One box writes and the other reads, as in the shared location of Section 2, and that model signals. What differs is what is written. Bob's outcome depends on Alice's *outcome*, so the boxes are not factored. Averaged over her outcome, however, it does not depend on her *setting*.[4]

To see this, write P_x for Alice's projector of outcome x on the pair, and Q_y for Bob's of outcome y. Her outcome x has probability $\langle\psi|P_x|\psi\rangle$. If that is positive, the outcome leaves $P_x|\psi\rangle$ normalized, in which Bob's outcome y has probability $\langle\psi|P_xQ_yP_x|\psi\rangle/\langle\psi|P_x|\psi\rangle$. The projectors are symmetric matrices; Alice's and Bob's act on different qubits, so $P_xQ_y = Q_yP_x$; and $P_xP_x = P_x$. The joint probability is therefore

$$[\, a, b \vdash x, y \,] = \langle\psi|P_xQ_y|\psi\rangle.$$

If $\langle\psi|P_x|\psi\rangle = 0$, Alice never answers x, and the formula still holds. Since $\langle\psi|P_x|\psi\rangle = \langle\psi|P_xP_x|\psi\rangle$ is the squared length of $P_x|\psi\rangle$, that vector is 0, and both sides are 0: the left because x never occurs, the right because it is the inner product of $P_x|\psi\rangle$ with $Q_y|\psi\rangle$. The formula is the same whichever box measures first: the two computations are dual in their effect as well as in their code. Each party's projectors add up to the identity, so summing over one party's outcome leaves the other's local response:

$$\sum_x \langle\psi|P_xQ_y|\psi\rangle = \langle\psi|Q_y|\psi\rangle, \qquad \sum_y \langle\psi|P_xQ_y|\psi\rangle = \langle\psi|P_x|\psi\rangle.$$

So the marginals of the joint response are the local responses: `fst <$> jointAB a b` has the distribution of `localA a`, and `snd <$> jointAB a b` that of `localB b`. These two equations say that neither party's outcome depends on the other's setting: the boxes are local. (They are blind, since the state is the same at every trial.)

All this holds for every state. A state monad could carry a signal; that this write carries none is a fact about projectors, not about types. The test suite checks these identities for several states and every context, and runs the pair with Bob's box first. A computation in `IO` does not expose its distribution, though, so we also check no-signaling on the outcomes of runs.

5.4 CHSH Behavior and No-Signaling

We run the maximally entangled pair under each schedule. Both runs come close to $2\sqrt{2} \approx 2.828$: 2.820 under the fixed schedule and 2.818 under the random one. The membrane ignores the step, so

[4] The names `nonlocalA` and `nonlocalB` say that these computations act on the state of the whole pair. The literature likewise calls correlations beyond the classical bound *nonlocal*. That sense is wider than our property *local*, which is parameter independence: the quantum boxes are nonlocal in the wide sense, and local in ours.

the choice of schedule has no effect. Neither run signals, and the table of the first is the empirical model of the quantum boxes:

a	b	$(+,+)$	$(+,-)$	$(-,+)$	$(-,-)$	$[\,x{=}+\,]$	$[\,y{=}+\,]$	trials
A0	B0	0.422	0.075	0.074	0.430	0.496	0.495	20000
A0	B1	0.424	0.074	0.075	0.427	0.497	0.499	20000
A1	B0	0.429	0.075	0.070	0.426	0.503	0.499	20000
A1	B1	0.074	0.430	0.423	0.073	0.504	0.497	20000

Every marginal is close to $1/2$, as it must be for $|\Phi^+\rangle$, so here the no-signaling check compares fair coins with fair coins. A partially entangled pair puts it to a stronger test. At $\theta = \pi/8$, each qubit on its own is more often 0 than 1, and its marginal depends on the observable measured:

a	b	$(+,+)$	$(+,-)$	$(-,+)$	$(-,-)$	$[\,x{=}+\,]$	$[\,y{=}+\,]$	trials
A0	B0	0.728	0.124	0.022	0.126	0.852	0.750	20000
A0	B1	0.727	0.125	0.021	0.127	0.852	0.748	20000
A1	B0	0.497	0.000	0.249	0.254	0.497	0.746	20000
A1	B1	0.249	0.249	0.502	0.000	0.498	0.751	20000

Alice answers `Plus` with frequency 0.852 at `A0` and 0.497 at `A1`, where the Born rule predicts $\cos^2\theta = 0.854$ and 0.500, and neither frequency moves with Bob's setting. Bob answers `Plus` with a frequency close to 0.750 at both of his settings, whatever Alice's. The run does not signal. Its CHSH value is 2.421, close to $\mathrm{CHSH}(\pi/8) = 1 + \sqrt{2} \approx 2.414$: a less entangled pair gives weaker correlations. The next section goes the other way, and replaces the quantum correlations with the strongest that no-signaling allows.

6 PR Boxes: The Algebraic Maximum without Signaling

The quantum boxes of Section 5 exceed the classical bound without signaling and without a hidden variable: they give up factoring, and stay blind and local. PR boxes (Popescu and Rohrlich, 1994) take this departure to its extreme. A PR box satisfies no-signaling exactly, and yet reaches the algebraic maximum 4 of the CHSH expression. Its correlations are stronger than any that quantum mechanics allows, although they permit no faster-than-light communication between the parties; for this reason PR boxes are often described as "super-quantum" or "post-quantum". In our framework, they arise from the same single freedom: the implementation of `jointAB`.

6.1 The PR Constraint

The defining property of a PR box is easiest to state in bits. Let the settings be bits $a, b \in \{0, 1\}$, and let the outcomes be bits $x, y \in \{0, 1\}$. The PR constraint is

$$x \oplus y = a \wedge b,$$

together with the requirement that x and y are each uniform. Translated back into outcomes ± 1, the constraint makes the outcomes agree in three contexts and disagree in the fourth, so all four CHSH terms contribute $+1$, for a total of 4. The two outcomes are perfectly correlated or anti-correlated, depending on the context, and yet neither party's marginal distribution depends on the other party's setting. The model is therefore exactly no-signaling.

6.2 A PR Trial Monad

Like the quantum boxes, PR boxes are most naturally written with genuine randomness. Their trial monad wraps `IO`:

```haskell
newtype PR a = PR { unPR :: IO a }                                    L48
  deriving newtype (Functor, Applicative, Monad, MonadIO)
```

PR trials, like quantum trials, have no memory and do not depend on the step, which the membrane discards:

```haskell
runTrialPR :: RunTrial IO PR                                          L49
runTrialPR (PR ioa) = ReaderT (\_ -> ioa)
```

The `TrialModel` instance says what the definition says. Each box, on its own, is a fair coin at either setting. The joint response runs the two boxes as the default does, and keeps only the pairs that meet the constraint: `m `suchThat` p` runs `m` again until its result satisfies `p`:

```haskell
instance TrialModel PR where                                          L50
  localA _ = PR randomOutcome
  localB _ = PR randomOutcome
  jointAB a b = liftA2 (,) (localA a) (localB b) `suchThat` prConstraint a b
prConstraint :: ASetting -> BSetting -> (Outcome, Outcome) -> Bool
prConstraint a b (x, y) = (x /= y) == (a == A1 && b == B1)
suchThat :: Monad m => m r -> (r -> Bool) -> m r
suchThat m p = do
  r <- m
  if p r then pure r else m `suchThat` p
```

The outcomes must differ exactly in context `(A1, B1)`, where $a \wedge b = 1$: that is the constraint $x \oplus y = a \wedge b$. The two boxes are dual, and neither sees the other's setting or outcome. All the correlation is in the constraint, which only the joint response can state, since only it has both settings.

Local, and as far from factored as can be. In each context two pairs of outcomes meet the constraint, equally likely since the coins are fair, and in those two each party answers `Plus` once. So each outcome is still a fair coin, at either of the other party's settings: the marginals of the joint response are the local responses, and the boxes are local. Given the settings, each outcome determines the other, so the boxes are as far from factored as they can be.

Keeping only some pairs is post-selection, which in a real Bell test is the detection loophole (Brunner et al., 2014). A PR box is hypothetical, and here the selection is its definition; the retries stay inside one trial, so the record of the run has no gaps.

The runs. Under each schedule, the run gives CHSH 4.000. The schedule cannot matter, because the correlations are defined context by context. And the reports confirm that the boxes do not signal: in both runs, every marginal is close to 1/2, whatever the other party's setting. Figure 8 tabulates the empirical model of the PR box; the table of each run is that model up to sampling fluctuations, and so was the table that `cheatNS` produced in Section 4.3.

Why fair coins? Unlike the quantum boxes, the PR box cannot have local responses that depend on the setting. The uniform marginals in its definition are no choice: any box that reaches CHSH 4 without signaling has fair coins for parts. Write p_a for the probability that Alice answers `Plus` at setting a, and q_b for Bob at b; without signaling, neither depends on the other party's setting.

a	b	$(+,+)$	$(+,-)$	$(-,+)$	$(-,-)$
A0	B0	$1/2$	0	0	$1/2$
A0	B1	$1/2$	0	0	$1/2$
A1	B0	$1/2$	0	0	$1/2$
A1	B1	0	$1/2$	$1/2$	0

Figure 8: The empirical model of the PR box.

CHSH 4 needs agreement in the contexts `(A0, B0)`, `(A0, B1)` and `(A1, B0)`, and disagreement in `(A1, B1)`. The three agreements give $p_0 = q_0$, $p_0 = q_1$ and $p_1 = q_0$, and the disagreement gives $p_1 = 1 - q_1$. Hence $p_1 = 1 - p_1$, and all four marginals are $1/2$.

The constraint does not supply this by itself: on coins biased to $3/4$, the same `jointAB` still reaches CHSH 4, but Alice's probability of `Plus` at `A1` is 0.900 next to `B0` and 0.500 next to `B1`. Local responses that read, and rewrite, a hidden variable do appear in Section 7.1, at the price of locality.

6.3 Quantum versus PR: Global Assignments

The quantum boxes and the PR box give up the same property in the same method. What, then, separates their tables, apart from the CHSH value? A value of type `Lambda` fixes an outcome for each of the four settings at once: it is a *global assignment*. The statistics of a run come from blind, local, factored boxes exactly when they come from a probability distribution over such values (Fine, 1982; Abramsky and Brandenburger, 2011). Neither the quantum boxes nor the PR box has such a distribution, since both exceed the classical bound. They differ in how far they are from having one.

With the maximally entangled pair, every one of the sixteen joint outcomes of the quantum boxes is possible (the rarest have frequency about 0.07 in the first table of Section 5.4), so each possible outcome lies on some global assignment; only the probabilities cannot be spread over the assignments. The PR box admits not even that: the outcomes it allows (Figure 8) fit no global assignment at all.

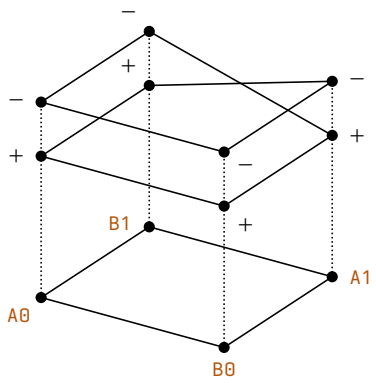


Figure 9: The PR box as a bundle. Over each setting stands a fiber of its two outcomes, and each edge between fibers is a pair of outcomes that the PR box allows in a context; the context `(A1, B1)` is joined crosswise.

Figure 9 draws the outcomes of the PR box as a bundle, with a fiber of two outcomes over each setting and an edge for each pair of outcomes that the PR box allows in a context. A global assignment would be a closed path that visits each fiber once: one outcome chosen in each fiber, such that all four chosen pairs are edges. The edges of the PR box form a discrete Möbius strip, with

the context `(A1, B1)` joined crosswise, and no such path exists. Abramsky and Brandenburger (2011) call this *strong contextuality* (see also Abramsky et al., 2015); quantum mechanics shows it only with more parties, as for GHZ states (Greenberger et al., 1990), or with more measurements, as in the Kochen–Specker theorem (Kochen and Specker, 1967).

The quantum boxes and the PR box thus differ only in how the two outcomes come to depend on each other in `jointAB`: through the state that a measurement leaves, by the Born rule, or by the PR constraint on the pair. Both are blind and local and do not signal, but the quantum boxes are bounded by $2\sqrt{2}$ and, at that bound, leave every outcome possible, while the PR box attains 4 and admits no global assignment.

7 Hidden States That the Boxes Rewrite

The quantum boxes and the PR box kept no hidden variable. The table of the PR box, and a table that quantum mechanics realizes, can also be produced with one, by boxes that stay factored and give up locality instead. The shared location of Section 2 did so crudely, and signaled. In this section the boxes share λ itself, and may update it during a trial (Figure 10): the definition of λ stays the same, but `Reader` becomes `State`. Each box's local response reads its entry of λ, as in Section 4.1, and then rewrites the other party's entries; the joint response keeps its default (L2). Whether such boxes signal will turn out to depend on their sampler.

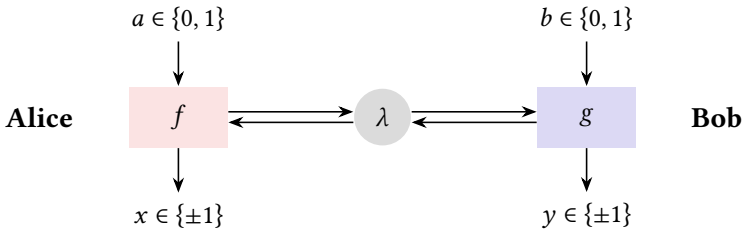


Figure 10: The nonlocal λ-model: the boxes read the hidden variable, and write to it.

7.1 The PR Box from a Hidden State

The trial monad is a state monad over λ:

```
newtype PRHiddenState a = PRHiddenState { unPRHiddenState :: State Lambda a }                L51
  deriving newtype (Functor, Applicative, Monad, MonadState Lambda)
```

Each box reads its own entry of λ, as in Section 4.1, and then overwrites both of the other party's entries: with its outcome if it measured its first setting, and with its outcome and its flip if it measured its second. `gets a0` reads the field `a0` of the state, and `modify` rewrites the state:

```
instance TrialModel PRHiddenState where                                                      L52
  localA A0 = do
    x ← gets a0
    modify (\lam → lam { b0 = x, b1 = x })
    pure x
  localA A1 = do
    x ← gets a1
    modify (\lam → lam { b0 = x, b1 = flipOutcome x })
    pure x
```

Bob's box is its dual:

```
localB B0 = do                                                    L53
  y <- gets b0
  modify (\lam -> lam { a0 = y, a1 = y })
  pure y
localB B1 = do
  y <- gets b1
  modify (\lam -> lam { a0 = y, a1 = flipOutcome y })
  pure y
```

The joint response keeps its default (L2), which runs Alice's box and then Bob's. Alice reads her outcome x from λ; Bob then reads an entry that Alice's box overwrote, so his outcome is $y = x$, except in context `(A1, B1)`, where it is the flip of x: the PR constraint. Bob's box rewrites Alice's entries in the same way, but she has already answered. The program has the shape of the shared location of Section 2, whose local responses also write, and not that of the quantum boxes, whose local responses only read and whose joint response is overridden (L45).

The membrane draws the initial λ with a sampler, as for local hidden variables; `evalState s` runs the trial from the state that `samp` draws:

```
runTrialPRHiddenState :: Functor e => Sampler e -> RunTrial e PRHiddenState   L54
runTrialPRHiddenState samp (PRHiddenState s) = evalState s <$> samp
```

We run the model with the cycling sampler under each schedule, and with the constant sampler `plus2` under the fixed schedule. With the cycling sampler, both runs give CHSH 4.000 and do not signal. (Under the fixed schedule the cycling sampler correlates Bob's entries of λ with the settings, as in Section 4.2, but Alice's box overwrites them before Bob reads them.) With `plus2`, the run still gives CHSH 4.000, but Bob at `B1` signals: his frequency of `Plus` there is 1.000 next to `A0` and 0.000 next to `A1`.

Why a uniform λ does not signal. The model is not local: given λ, Bob's outcome depends on Alice's setting, through the entries that Alice's box overwrote. (It is trivially factored, since the settings and λ determine both outcomes.) In bits, Bob's outcome is $y = x \oplus (a \wedge b)$, where x is Alice's entry of λ. When λ is uniform, x is a fair coin, and so is y, whatever a is: Alice's setting reaches Bob only masked by a uniformly random bit, as in a one-time pad, and Bob's marginal cannot depend on it. With `plus2`, x is always `Plus`, the mask is gone, and Bob's outcome at `B1` reveals Alice's setting. The shared location of Section 2 is the same channel without the mask.

No-signaling here is a property of the sampler, not of the boxes. The quantum boxes also write to a state during a trial, and have the opposite properties: they are local and not factored. Their local responses only read, and only their joint response writes; their outcomes are drawn in the measurement rather than fixed by λ; and they do not signal from any state. There, no-signaling is a property of the boxes.

7.2 The Bell Table from a Hidden State

A hidden state that the boxes rewrite can also produce correlations weaker than the PR box's, among them correlations that quantum mechanics realizes. Figure 11 shows the Bell table of Abramsky (2017): the outcomes always agree in context `(A0, B0)`, agree three times in four in `(A0, B1)` and `(A1, B0)`, and agree once in four in `(A1, B1)`. Its CHSH value is $1 + \frac{1}{2} + \frac{1}{2} + \frac{1}{2} = 2.5$, above the classical bound 2 and below the Tsirelson bound $2\sqrt{2}$. The quantum boxes of Section 5 realize it, with

other observables: in the state $|\Phi^+\rangle$, with Alice measuring Z or $(Z+\sqrt{3}\,X)/2$ and Bob measuring Z or $(Z-\sqrt{3}\,X)/2$, the outcomes agree with probability 1, 3/4, 3/4 and 1/4. (The accompanying code computes these four probabilities by the Born rule on the pair.)

a	b	$(+,+)$	$(+,-)$	$(-,+)$	$(-,-)$
A0	B0	1/2	0	0	1/2
A0	B1	3/8	1/8	1/8	3/8
A1	B0	3/8	1/8	1/8	3/8
A1	B1	1/8	3/8	3/8	1/8

Figure 11: The Bell table (Abramsky, 2017, Figure 3).

The boxes. The trial monad is again a state monad over λ:

```
newtype BellHiddenState a = BellHiddenState { unBellHiddenState :: State Lambda a }
  deriving newtype (Functor, Applicative, Monad, MonadState Lambda)
```
L55

As for the PR box, each box reads its own entry of λ and then rewrites the other party's entries, and the joint response keeps its default. To agree three times in four rather than always, a box copies its outcome into the other party's entries or flips it, by coins that it finds in λ itself. Alice's box:

```
instance TrialModel BellHiddenState where
  localA A0 = do
    x <- gets a0
    modify (\lam -> lam { b0 = x, b1 = flipWhen (rare (a1 lam) (b1 lam)) x })
    pure x
  localA A1 = do
    x <- gets a1
    modify (\lam -> lam { b0 = flipWhen (rare (a0 lam) (b0 lam)) x
                        , b1 = flipWhen (not (rare (a0 lam) (b1 lam))) x })
    pure x
```
L56

Bob's box is its dual:

```
  localB B0 = do
    y <- gets b0
    modify (\lam -> lam { a0 = y, a1 = flipWhen (rare (b1 lam) (a1 lam)) y })
    pure y
  localB B1 = do
    y <- gets b1
    modify (\lam -> lam { a0 = flipWhen (rare (b0 lam) (a0 lam)) y
                        , a1 = flipWhen (not (rare (b0 lam) (a1 lam))) y })
    pure y
```
L57

Once a box has read its entry, the other three entries of a uniform λ are still fair coins, independent of its outcome, and two of them are both `Minus` a quarter of the time. Such a pair is a coin, and we say that it *comes up* when `rare` holds of it:

```
rare :: Outcome -> Outcome -> Bool
rare u v = u == Minus && v == Minus
flipWhen :: Bool -> Outcome -> Outcome
flipWhen c x = if c then flipOutcome x else x
```
L58

In the order of the joint response, Bob's outcome is x in context `(A0, B0)`; x flipped when the coin comes up in `(A0, B1)` and `(A1, B0)`; and x flipped unless the coin comes up in `(A1, B1)`.

In bits, $y = x \oplus e$, where e is 1 with probability 0, 1/4, 1/4 and 3/4 in the four contexts. Nothing reads the state after Bob's write, but either box could go first: Alice's outcome would then be Bob's, flipped with the same probabilities, and a uniform λ gives the same table.

As in Section 7.1, no-signaling is a property of the sampler: when λ is uniform, the first outcome is a fair coin, the second is the first, flipped by an independent coin, and both marginals are 1/2 whatever the other party's setting. The membrane is the one of that section:

```
runTrialBellHiddenState :: Functor e => Sampler e -> RunTrial e BellHiddenState                L59
runTrialBellHiddenState samp (BellHiddenState s) = evalState s <$> samp
```

The runs. The coins are fair only if every context meets a uniform λ, which the cycling sampler under the fixed schedule does not provide (Section 4.2). A sampler that keeps each strategy for a block of four steps, as `biasPlus2Blocks` does, lets every context meet all sixteen equally often:

```
uniformBlocks :: Applicative e => Sampler e                                                    L60
uniformBlocks = ReaderT (\i -> uniformCycle allLambdas (i `div` 4))
```

With `uniformBlocks` under the fixed schedule, the run's table is the Bell table, count for count, with CHSH 2.500 and no signaling:

a	b	$(+,+)$	$(+,-)$	$(-,+)$	$(-,-)$	$[x{=}+]$	$[y{=}+]$	trials
A0	B0	0.500	0.000	0.000	0.500	0.500	0.500	20000
A0	B1	0.375	0.125	0.125	0.375	0.500	0.500	20000
A1	B0	0.375	0.125	0.125	0.375	0.500	0.500	20000
A1	B1	0.125	0.375	0.375	0.125	0.500	0.500	20000

The test suite also runs the pair with Bob's box first, and gets the same table, count for count. With the cycling sampler under the random schedule, the run gives CHSH 2.508 and does not signal.

Under the fixed schedule, the cycling sampler fixes Bob's entries of λ with the context, as in Section 4.2, and the coins with them: `b1` is always `Plus` in contexts `(A0, B1)` and `(A1, B1)`, and so is `b0` in `(A1, B0)`. No coin ever comes up, so Bob's outcome copies Alice's except in `(A1, B1)`, where it flips it, and the run is the PR box: CHSH 4.000, without signaling. The boxes are the same in all three runs; the sampler and the schedule decide which table they produce, and the table shows it.

A logical Bell inequality. In the terms of Section 6.3, the Bell table behaves like the table of the quantum boxes. The outcomes it allows fit eight global assignments, those with equal entries for `A0` and `B0`, and each allowed outcome lies on one of them. Its CHSH value rules out a distribution over them. Abramsky (2017) reads this as a logical Bell inequality: no global assignment makes all four events hold at once, the three agreements and the disagreement in `(A1, B1)`, so their probabilities sum to at most 3; in the Bell table they sum to $1 + \frac{3}{4} + \frac{3}{4} + \frac{3}{4} = 3.25$.

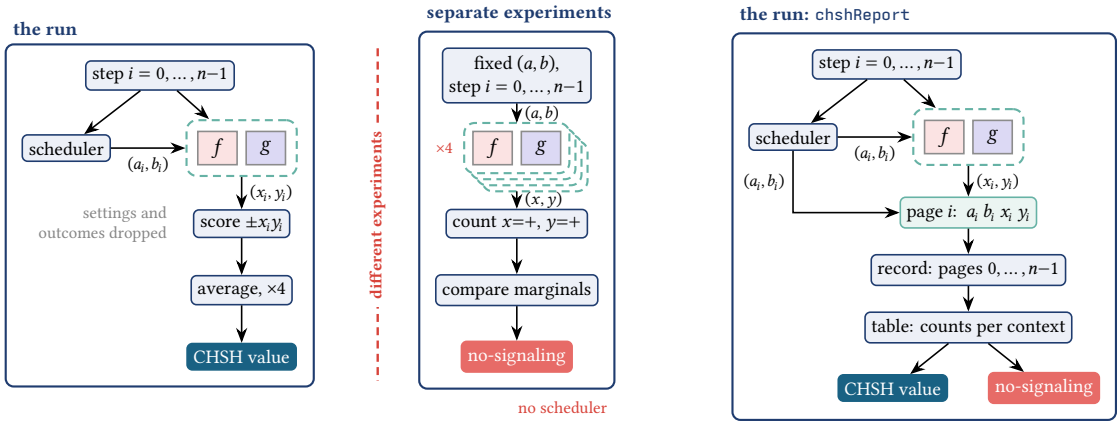
A hidden variable that the boxes may rewrite thus reaches a quantum table as easily as the PR box. What Bell's theorem rules out is a hidden variable that neither box can change, as in Section 4.1.

8 No-Signaling, Checked in the Run

Every verdict so far was read off the table of the run that produced the CHSH value. This section shows why that matters, by comparing it with the obvious alternative: checking no-signaling in experiments of its own.

8.1 Two Ways to Check

Equation (2) compares conditional probabilities at fixed settings. It is tempting to estimate them in separate experiments at fixed settings: fix (a, b), run the trial at the steps $0, \ldots, n-1$, and count the outcomes. No scheduler is involved, and the CHSH value comes from another run.[5] Figure 12 sets this check beside the one of Section 3.4 (L19), in which both verdicts come from one table.



(a) A separate check: the verdict comes from other experiments.

(b) One run, one table, both verdicts.

Figure 12: Two ways to check no-signaling. A separate check fixes the settings in advance and never sees the schedule; in a run, each step writes a page, and both verdicts are read off the same table.

8.2 Why the Two Can Differ

Write $[\, a, b \vdash y \,]_i$ for the probability of Bob's outcome y at step i when the settings are (a, b), and recall that $[\, a, b \,]_i$ is the probability that the scheduler picks (a, b) at step i. The run's table averages over the steps that measured (a, b); the separate check fixes (a, b) and uses every step once:

$$[\, a, b \vdash y \,]_{\text{run}} = \frac{\sum_i [\, a, b \,]_i \, [\, a, b \vdash y \,]_i}{\sum_i [\, a, b \,]_i}, \qquad [\, a, b \vdash y \,]_{\text{check}} = \frac{1}{n} \sum_i [\, a, b \vdash y \,]_i. \tag{4}$$

The two agree when $[\, a, b \vdash y \,]_i$ is the same at every step, as for the quantum and PR boxes, whose membranes ignore the step, or when $[\, a, b \,]_i$ is, as under the random schedule. They can differ when both change with i: under the fixed schedule, with a membrane that passes the step on to the boxes, or with a sampler that reads it. The same holds for Alice's outcome x.

The clocked boxes with the clock visible are an example. For them, $[\, a, b \vdash y{=}+ \,]_i$ is 0 when $i \bmod 4 = 3$ and 1 otherwise, whatever (a, b). The separate check therefore sees Bob answer `Plus` at `B1` with frequency 3/4 next to either of Alice's settings (0.750 and 0.750 in our run of it), and it passes. Under the fixed schedule, the run measures `(A1, B1)` exactly at the steps where Bob answers `Minus`, and finds 1.000 next to `A0` and 0.000 next to `A1`. Of the two experiments, only the run is the one whose CHSH value was reported.

[5] An earlier version of this paper checked no-signaling in this way, with 20000 trials per context and a tolerance of 0.02 on each difference of frequencies.

Model	Sampler	Schedule	Seed	CHSH	In the run	Separate
Pure boxes	–	fixed	–	2.000	✓	✓
Pure boxes	–	random	0	2.000	✓	✓
Shared location	–	fixed	–	4.000	✗ Bob at `B1`	✗
Clocked, clock visible	–	fixed	–	4.000	✗ Bob at `B1`	✓
Clocked, clock visible	–	random	2	0.990	✓	✓
Clocked, clock hidden	–	fixed	–	2.000	✓	✓
Clocked, clock hidden	–	random	3	2.000	✓	✓
Local hidden variables	`plus2`	fixed	–	2.000	✓	✓
Local hidden variables	`minus2`	fixed	–	−2.000	✓	✓
Local hidden variables	`uniformAll`	fixed	–	0.000	✗ Bob at `B0`	✓
Local hidden variables	`uniformAll`	random	1	0.008	✓	✓
Local hidden variables	`biasPlus2Loophole 7 8`	fixed	–	3.000	✗ Bob at `B1`	✓
Local hidden variables	`biasPlus2Blocks 7 8`	fixed	–	1.500	✓	✓
Local hidden variables	`fresh uniformIO`	fixed	13	0.024	✓	✓
Superdeterminism	`cheatLam`	fixed	–	4.000	✗ Bob at `B1`	–
Superdeterminism	`cheatLam`	random	4	4.000	✗ Bob at `B1`	–
Superdeterminism	`legalAllPlus`	fixed	–	2.000	✓	–
Superdeterminism	`cheatNS`	random	10	4.000	✓	–
LHV, sampler peeks	`peek fixedSchedule cheatLam`	fixed	–	4.000	✗ Bob at `B1`	✓
LHV, sampler peeks	`peek randomScheduleIO cheatLam`	random	4	1.990	✓	✓
Quantum	–	fixed	5	2.820	✓	✓
Quantum	–	random	6	2.818	✓	✓
Quantum, $\theta = \pi/8$	–	fixed	14	2.421	✓	✓
PR box	–	fixed	7	4.000	✓	✓
PR box	–	random	8	4.000	✓	✓
PR hidden state	`uniformAll`	fixed	–	4.000	✓	✓
PR hidden state	`uniformAll`	random	11	4.000	✓	✓
PR hidden state	`plus2`	fixed	–	4.000	✗ Bob at `B1`	✗
Bell table, hidden state	`uniformBlocks`	fixed	–	2.500	✓	✓
Bell table, hidden state	`uniformAll`	random	15	2.508	✓	✓
Bell table, hidden state	`uniformAll`	fixed	–	4.000	✓	✓

Figure 13: Every run of this paper, 80000 trials each: the CHSH value and the no-signaling verdict of its own table, and the verdict of a separate check on the same membrane (20000 trials per context). Seed: the seed of a run in `IO` (L9); a run in `Identity` draws nothing at random. The separate checks in `IO` have seeds of their own, in the accompanying code. ✗ marks a run that signals, with the condition that its table violates. The shaded runs signal, and pass the separate check. Superdeterministic runs have no separate check.

8.3 Every Model, Both Checks

Figure 13 lists every run of this paper, with the verdict of its own table and that of the separate check on the same membrane. The separate check passes four runs whose own table signals: the clocked boxes with the clock visible, the cycling sampler and the biased sampler of Section 4.2, and the sampler that peeks, all under the fixed schedule. In all four, the step correlates the settings with the behavior of the boxes. The separate check never sees the schedule, so it never sees the correlation,

and its verdict is the same under either schedule; the verdict of the run depends on the schedule, as it must.

The two checks agree on every other run that has both: on the shared location and the PR box from a hidden state with `plus2`, which signal within a trial, and on the quantum and PR boxes, whose membranes ignore the step.

Superdeterministic runs have no entry in the last column. Their membrane depends on the settings, and a separate check must choose which settings to hand it. Our earlier version handed it the scheduler's settings at each step, by running the scheduler a second time inside the membrane. That membrane is `runTrialLHV (peek fixedSchedule cheatLam)`, whose run signals while its separate check passes; and under a random schedule it loses the cheat altogether (CHSH 1.990), because the second run of the scheduler flips new coins. In a run, no such choice arises: the trial hands the membrane the settings it drew.

9 Conclusion

We conclude by collecting the models of this paper, in Figure 14, by the three properties of Section 4 and by the verdicts of their runs.

Model	Blind	Local	Factored	No-signaling	CHSH
Local hidden variables (§4.1)	✓	✓	✓	✓	≤ 2
Not blind					
Scheduler-induced loopholes (§3.5, §4.2)	✗	✓	✓	✗	≤ 4
Superdeterminism (§4.3)	✗	✓	✓	✗ / ✓	≤ 4
Not factored					
Quantum boxes (§5)	✓	✓	✗	✓	$\leq 2\sqrt{2}$
PR boxes (§6)	✓	✓	✗	✓	4
Not local					
Shared location (§2)	✓	✗	✓	✗	≤ 4
PR box from a hidden state (§7.1)	✓	✗	✓	✓ / ✗	4
Bell table from a hidden state (§7.2)	✓	✗	✓	✓	2.5 / 4

Figure 14: The models of this paper, by the property that each gives up. The no-signaling column is the verdict of each model's own runs (Figure 13): the scheduler-induced loopholes signal under the fixed schedule, a superdeterministic sampler may move a marginal (`cheatLam`) or keep both fair (`cheatNS`), and the PR box from a hidden state does not signal with the cycling sampler but does with `plus2`. The Bell table from a hidden state gives 2.5 when every context meets a uniform λ, and the PR box's 4 with the cycling sampler under the fixed schedule, a run that is not blind either (Section 4.2).

By rows: a hierarchy. Local hidden variables have all three properties and stay within 2. The scheduler-induced loopholes and superdeterminism give up blindness, and the shared location gives up locality; all three reach 4 for reasons that have nothing to do with quantum theory. Quantum and PR boxes give up only factoring: they stay blind and local, do not signal, and reach $2\sqrt{2}$ and 4. The hidden states of Section 7, like the shared location, give up locality, and do not signal only

because their sampler masks the channel between the boxes. They reach the 4 of the PR box and, with coins that the boxes find in λ itself, a quantum value: the 2.5 of the Bell table.

By columns: three statements about a program. Each property says something about one place in the code.

- **Blind:** `liftA2 (,) sched samp`, in the experiment layer (L30). The settings and the strategy of a step form an applicative pair, in which the sampler does not see what the scheduler returns.
- **Factored:** `liftA2 (,) (localA a) (localB b)`, in the trial layer (L2). The two outcomes of a trial form an applicative pair, the default `jointAB`, in which neither box sees what the other returns.
- **Local:** `localA :: ASetting → t Outcome`, and likewise `localB`. Each box is a function of its own setting, in an effect `t` through which the other setting cannot reach it: a reader monad lets the boxes read λ but not talk.

Blind and factored thus have the same shape at the two layers, and giving up either makes the two components of the pair depend on each other. Superdeterminism replaces the pair by a bind, which draws the strategy from the settings (L35). The boxes that give up factoring keep `liftA2`, and the dependence comes in another way. The quantum boxes collapse a shared state (L46): in a monad, `liftA2` runs one computation after the other, so the dependence can travel through the effect. The PR box keeps only the pairs that meet its constraint (L50), so the dependence comes from outside the pair. Local is what keeps that dependence from signaling: Bob's outcome may depend on Alice's outcome, but, averaged over it, not on her setting.

What the code does not enforce. None of the three properties is guaranteed by types. A sampler of the legal type can follow the settings through the step that it shares with the scheduler (Section 4.2). A state monad hides a channel behind local types, in the shared location (Section 2) and in the boxes with a hidden state (Section 7), whose joint response is even the default. The quantum boxes write to a state too, but only in their joint response, and that this write carries no signal is a fact about projectors, not about types (Section 5.3). The properties belong to what a program does, not to what it looks like. That is why the verdicts are read off its runs; and no-signaling in particular is a property of a run, not of the boxes alone (Section 8).

Who goes first. In a state monad, `liftA2` runs Alice's box and then Bob's, and the second sees the effect of the first, though not its value. For the quantum boxes and the PR box the order does not show: their joint distribution is the same either way (Section 5.3; the PR box draws two independent coins, and its constraint treats both parties alike). For the boxes with a hidden state it shows, and must: the box that goes first answers from its own setting and λ alone, so if the two orders always agreed, both boxes would be local, and with a blind sampler held to 2. Their no-signaling holds in both orders when every context meets a uniform λ, as under the random schedule; otherwise it may depend on the order. The cycling sampler under the fixed schedule pins Bob's entries of λ (Section 4.2), which Alice's box overwrites when it goes first. With Bob's box first, both runs of Section 7 with that sampler and schedule signal, and the Bell boxes fall from CHSH 4.000 to 2.000. Their verdicts in Figure 13 hold for Alice's box first: the order of evaluation, which has no physical counterpart, is part of the run too.

That is the contribution. The Bell experiment is a small program whose behavior is determined by where and how information is allowed to flow—what each part sees, and how the parts are composed—and whose two verdicts come from the record of one run.

Bell's theorem and contextuality are often presented as metaphysical claims about the nature of reality. At a technical level, both concern which local behaviors admit a global explanation, and under which independence assumptions. The framework does not resolve the foundational questions, nor does it reduce quantum mechanics to classical computation; it makes Bell's assumptions visible, and testable, in the structure of a program.

10 Related Work

Bell's assumptions. Bell (1964) introduced local hidden-variable theories, and Clauser et al. (1969) turned his argument into the inequality that experiments test, from Aspect et al. (1982) to the loophole-free test of Hensen et al. (2015). Our three properties are the standard assumptions behind it. Blindness is λ-independence, also called measurement independence (Brunner et al., 2014). Local and factored are the two conditions into which Jarrett (1984) split Bell's locality, and which he called locality and completeness; Shimony (1984) named them parameter independence and outcome independence. Quantum mechanics keeps parameter independence and violates outcome independence, which in our code is the decision to override `jointAB` with measurements that collapse the state of the pair. The hidden states of Section 7 keep the default `jointAB` and let each box's local response rewrite λ; they violate parameter independence instead: the settings and λ determine both outcomes, so outcome independence holds trivially, and Bob's outcome depends on Alice's setting. Brandenburger and Yanofsky (2008) classify these properties and the relations among them. Superdeterminism (Hossenfelder and Palmer, 2020; Palmer, 2009; 't Hooft, 2016) and the free will theorems (Conway and Kochen, 2006, 2009) concern the first.

No-signaling and contextuality. Popescu and Rohrlich (1994) showed that no-signaling alone permits a CHSH value of 4, which started a search for principles that single out the quantum correlations (Brunner et al., 2014). Abramsky and Brandenburger (2011) treat nonlocality and contextuality alike: an empirical model is a family of distributions, one per context, that agree on overlaps—in a Bell scenario, this agreement is no-signaling—and contextuality is the absence of a global section, which here is a distribution over values of type `Lambda` that gives every context its own (see also Abramsky, 2013; Abramsky et al., 2015). The tables of our runs are empirical models in a weaker sense: nothing makes them agree on overlaps, and whether they do is one of the two verdicts. Specker (1960) and Kochen and Specker (1967) found contextuality without spatial separation.

Causal models. Causal analyses of Bell's theorem derive the inequalities from causal graphs and conditional independences, in the framework of Pearl (2009). We read the causal structure off the program instead: the step that the scheduler and the sampler share is a common cause, and superdeterminism is an applicative pair refactored into a bind.

Quantum programming. The Quantum IO monad (Altenkirch and Green, 2010) computes with quantum states in Haskell. A trial model built on it would compute the statistics of any quantum experiment. Ours measures the state vector of two qubits as a textbook does (Nielsen and Chuang, 2010), which suffices for the CHSH experiment.

11 Future Directions

Quantitative relaxations. Partial measurement dependence or limited signaling can be modeled by controlled dependencies between the sampler, the scheduler and the boxes. Seeing how CHSH values degrade along such paths may clarify the boundary between classical and non-classical regimes in operational terms.

Other effects. The present work uses `Reader`, `State` and `IO`. Continuations and concurrency primitives offer other ways to express dependencies between local and joint behavior, and may reveal correspondences with interpretations of quantum mechanics such as pilot-wave theory (Bohm, 1952), the transactional interpretation (Cramer, 1986) and many-worlds (Everett, 1957).

Deriving the Tsirelson bound. Our quantum model derives its correlations from the Born rule, and attains $2\sqrt{2}$ with observables chosen for it. That no quantum experiment does better is Tsirelson's theorem (Tsirelson, 1980), which the code does not prove. A trial model built on the Quantum IO monad (Altenkirch and Green, 2010) would cover every quantum experiment, not only measurements of two qubits; better still would be a constraint on joint composition, perhaps reflected in a refined monadic structure, that entails the bound.

No-signaling by construction. We check no-signaling on runs because the code does not enforce it. Pairing two computations with `liftA2` does not stop them from communicating through the effect they share (Section 2), and a run can signal through the experiment layer even when the boxes cannot (Sections 3.5 and 4.2). Enforcing it requires resources that the parties cannot share, as in the tensor of linear logic (Girard, 1987) or the separating conjunction of separation logic (Reynolds, 2002); no-signaling has been characterized equationally in categorical and linear-logic settings (Kissinger and Uijlen, 2017; Lindenhovius and Zamdzhiev, 2025). A type for it would have to constrain the boxes together with the experiment layer that runs them.

Qubits as computations, and beyond CHSH. The type of a local response, Setting $\longrightarrow$ t Outcome, records that a measurement does not reveal a value fixed in advance: an outcome is produced by interaction, inside an effect. Multipartite models such as GHZ experiments (Greenberger et al., 1990) and multi-setting ones such as Mermin's device (Mermin, 1981) would add an applicative layer f of separated parties, f (Setting $\longrightarrow$ t Outcome), and the same construction could instantiate restricted toy theories such as that of Spekkens (2007).

AI Use

An AI assistant, Claude (Anthropic), was used in preparing this paper and its artifact. It wrote and revised parts of the Haskell package, its test suite, and the program that generates the paper's numbers and tables. It checked claims of the text by running the code, and drafted and edited parts of the text, the figures and Appendix A. The author directed this work and reviewed its results. The artifact guards what the paper reports: every listing is quoted from the code by line number, every number, table and printed report comes from the data program, which stops if a claim of the text about the runs does not hold, and the test suite passes.

References

Samson Abramsky. Relational hidden variables and non-locality. *Studia Logica*, 101(2):411–452, 2013. doi: 10.1007/s11225-013-9477-4.

Samson Abramsky. Contextuality: At the borders of paradox. In Elaine Landry, editor, *Categories for the Working Philosopher*, pages 262–285. Oxford University Press, 2017. arXiv:2011.04899.

Samson Abramsky and Adam Brandenburger. The sheaf-theoretic structure of non-locality and contextuality. *New Journal of Physics*, 13:113036, 2011. doi: 10.1088/1367-2630/13/11/113036.

Samson Abramsky, Rui Soares Barbosa, Kohei Kishida, Raymond Lal, and Shane Mansfield. Contextuality, cohomology and paradox. In Stephan Kreutzer, editor, *24th EACSL Annual Conference on Computer Science Logic (CSL 2015)*, volume 41 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 211–228. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2015. doi: 10.4230/LIPIcs.CSL.2015.211.

Thorsten Altenkirch and Alexander S. Green. The quantum IO monad. In Simon Gay and Ian Mackie, editors, *Semantic Techniques in Quantum Computation*, pages 173–205. Cambridge University Press, 2010.

Alain Aspect, Philippe Grangier, and Gérard Roger. Experimental realization of Einstein–Podolsky–Rosen–Bohm Gedankenexperiment: A new violation of Bell's inequalities. *Physical Review Letters*, 49(2):91–94, 1982. doi: 10.1103/PhysRevLett.49.91.

John S. Bell. On the Einstein Podolsky Rosen paradox. *Physics Physique Fizika*, 1(3):195–200, 1964. doi: 10.1103/PhysicsPhysiqueFizika.1.195.

David Bohm. A suggested interpretation of the quantum theory in terms of "hidden" variables. I and II. *Physical Review*, 85(2):166–193, 1952.

Adam Brandenburger and Noson Yanofsky. A classification of hidden-variable properties. *Journal of Physics A: Mathematical and Theoretical*, 41(42):425302, 2008. doi: 10.1088/1751-8113/41/42/425302.

Nicolas Brunner, Daniel Cavalcanti, Stefano Pironio, Valerio Scarani, and Stephanie Wehner. Bell nonlocality. *Reviews of Modern Physics*, 86(2):419–478, 2014. doi: 10.1103/RevModPhys.86.419.

John F. Clauser, Michael A. Horne, Abner Shimony, and Richard A. Holt. Proposed experiment to test local hidden-variable theories. *Physical Review Letters*, 23(15):880–884, 1969. doi: 10.1103/PhysRevLett.23.880.

John Conway and Simon Kochen. The free will theorem. *Foundations of Physics*, 36(10):1441–1473, 2006. doi: 10.1007/s10701-006-9068-6.

John H. Conway and Simon B. Kochen. The strong free will theorem. *Notices of the AMS*, 56(2):226–232, 2009.

John G. Cramer. The transactional interpretation of quantum mechanics. *Reviews of Modern Physics*, 58(3): 647–687, 1986. doi: 10.1103/RevModPhys.58.647.

Hugh Everett. "Relative State" formulation of quantum mechanics. *Reviews of Modern Physics*, 29(3):454–462, 1957. doi: 10.1103/RevModPhys.29.454.

Arthur Fine. Hidden variables, joint probability, and the Bell inequalities. *Physical Review Letters*, 48(5): 291–295, 1982. doi: 10.1103/PhysRevLett.48.291.

Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–101, 1987. doi: 10.1016/0304-3975(87)90045-4.

Daniel M. Greenberger, Michael A. Horne, Abner Shimony, and Anton Zeilinger. Bell's theorem without inequalities. *American Journal of Physics*, 58(12):1131–1143, 1990.

Bas Hensen, Hannes Bernien, Anaïs E. Dréau, Andreas Reiserer, Norbert Kalb, Machiel S. Blok, Just Ruitenberg, Raymond F. L. Vermeulen, Raymond N. Schouten, Carlos Abellán, Waldimar Amaya, Valerio Pruneri, Morgan W. Mitchell, Matthew Markham, Daniel J. Twitchen, David Elkouss, Stephanie Wehner, Tim H. Taminiau, and Ronald Hanson. Loophole-free Bell inequality violation using electron spins separated by 1.3 kilometres. *Nature*, 526:682–686, 2015. doi: 10.1038/nature15759.

Sabine Hossenfelder and Tim Palmer. Rethinking superdeterminism. *Frontiers in Physics*, 8:139, 2020. doi: 10.3389/fphy.2020.00139.

Graham Hutton. *Programming in Haskell*. Cambridge University Press, 2nd edition, 2016.

Jon P. Jarrett. On the physical significance of the locality conditions in the Bell arguments. *Noûs*, 18(4): 569–589, 1984.

Aleks Kissinger and Sander Uijlen. A categorical semantics for causal structure. In *32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 1–12, 2017. doi: 10.1109/LICS.2017.8005095.

Simon Kochen and Ernst P. Specker. The problem of hidden variables in quantum mechanics. *Journal of Mathematics and Mechanics*, 17(1):59–87, 1967.

Bert Lindenhovius and Vladimir Zamdzhiev. Operator spaces, linear logic and the Heisenberg–Schrödinger duality of quantum theory. In *40th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 870–883. IEEE Computer Society, 2025. doi: 10.1109/LICS65433.2025.00071.

N. David Mermin. Quantum mysteries for anyone. *The Journal of Philosophy*, 78(7):397–408, 1981. doi: 10.2307/2026482.

Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition, 2010. doi: 10.1017/CBO9780511976667.

Timothy N. Palmer. The invariant set postulate: A new geometric framework for the foundations of quantum theory and the role played by gravity. *Proceedings of the Royal Society A*, 465(2110):3165–3188, 2009. doi: 10.1098/rspa.2009.0080.

Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

Sandu Popescu and Daniel Rohrlich. Quantum nonlocality as an axiom. *Foundations of Physics*, 24(3):379–385, 1994. doi: 10.1007/BF02058098.

John C. Reynolds. Separation logic: A logic for shared mutable data structures. In *17th Annual IEEE Symposium on Logic in Computer Science (LICS)*, pages 55–74, 2002. doi: 10.1109/LICS.2002.1029817.

Abner Shimony. Controllable and uncontrollable non-locality. In *Proceedings of the International Symposium on Foundations of Quantum Mechanics in the Light of New Technology*, pages 225–230. Physical Society of Japan, 1984.

Ernst Specker. Die Logik nicht gleichzeitig entscheidbarer Aussagen. *Dialectica*, 14(2–3):239–246, 1960.

Robert W. Spekkens. Evidence for the epistemic view of quantum states: A toy theory. *Physical Review A*, 75 (3):032110, 2007. doi: 10.1103/PhysRevA.75.032110.

Gerard 't Hooft. *The Cellular Automaton Interpretation of Quantum Mechanics*, volume 185 of *Fundamental Theories of Physics*. Springer, 2016. doi: 10.1007/978-3-319-41285-6.

Boris S. Tsirelson. Quantum generalizations of Bell's inequality. *Letters in Mathematical Physics*, 4(2):93–100, 1980. doi: 10.1007/BF00417500.

A Quantum Mechanics from Scratch

This appendix introduces the quantum mechanics that Section 5 uses, assuming none: one qubit and its measurements (Sections A.1 and A.2), two qubits and entanglement (Section A.3), and the correlations that the CHSH value combines (Section A.4). It needs only vectors and matrices, and it keeps to real numbers, which suffice for every state and measurement in this paper.[6] Figure 16, at the end, lists each notion with its name in the code. A textbook such as Nielsen and Chuang (2010) goes much further.

A.1 One Qubit

A bit is 0 or 1. A *qubit* has a *state*: a vector of two real numbers, its *amplitudes*, whose squares add up to 1,

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = [\alpha, \beta], \qquad \alpha^2 + \beta^2 = 1,$$

where $|0\rangle = [1, 0]$ and $|1\rangle = [0, 1]$ are its two *basis states*. The bracket $|\cdot\rangle$, a *ket*, just marks a vector; the notation is Dirac's. A state is thus a unit vector in the plane (Figure 15a). Flipping the sign of the whole vector changes nothing: $-|\psi\rangle$ is the same state as $|\psi\rangle$, because every probability below is the square of an amplitude. Flipping the sign of one amplitude does change the state, as $|+\rangle$ and $|-\rangle$ below show. So a state is a line through the origin, which we call a *direction*.

Measuring a qubit gives one of two outcomes, at random. Measured in its basis, a qubit in the state $|\psi\rangle$ gives 0 with probability α^2 and 1 with probability β^2, and afterwards it is in the basis state that was found: measuring it again in the same way gives the same outcome. The state fixes only the probabilities. A state with both amplitudes nonzero is a *superposition* of $|0\rangle$ and $|1\rangle$. For example, each of

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \qquad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

gives 0 and 1 with probability $1/2$. They are nevertheless different states, and the measurement X of Section A.2 tells them apart with certainty: a qubit is not a biased coin.

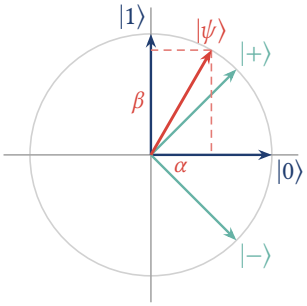
Two vectors have an *inner product*, $\langle u|v\rangle = u_0v_0 + u_1v_1$, written `u <| v` in the code. The length of $|v\rangle$ is $\sqrt{\langle v|v\rangle}$, two vectors are perpendicular when their inner product is 0, and the amplitude α of $|\psi\rangle$ is $\langle 0|\psi\rangle$, its component along $|0\rangle$.

A.2 Measurements: Observables and Projectors

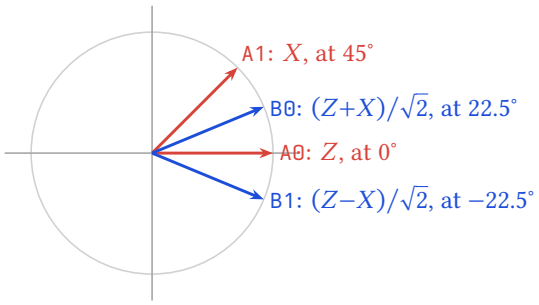
Measuring in the basis $|0\rangle$, $|1\rangle$ is one measurement among many. Any two perpendicular unit vectors $|v_+\rangle$ and $|v_-\rangle$ define one, with the outcomes +1 and −1, by the same rule. Write the state in terms of them, $|\psi\rangle = c_+|v_+\rangle + c_-|v_-\rangle$: the outcome is +1 with probability c_+^2 and −1 with probability c_-^2, and afterwards the qubit is in $|v_+\rangle$ or $|v_-\rangle$ accordingly. Figure 15a shows two such pairs, $|0\rangle$, $|1\rangle$ and $|+\rangle$, $|-\rangle$.

Observables. Physics names a measurement by a matrix, its *observable*. An *eigenvector* of a matrix O is a nonzero vector that O only scales, $O|v\rangle = \lambda|v\rangle$, and λ is its *eigenvalue*. The observable of the measurement above is the matrix with the eigenvectors $|v_+\rangle$ and $|v_-\rangle$, and the eigenvalues +1 and

[6] In general, amplitudes are complex numbers, an observable is a Hermitian matrix, and a probability is the squared absolute value of an amplitude. With real numbers these become what we use here: real amplitudes, symmetric matrices, and squares.



(a) A state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, and the two measurements Z (along $|0\rangle$ and $|1\rangle$) and X (along $|+\rangle$ and $|-\rangle$).



(b) The four measurements of Section 5.1, each drawn as its direction for +1; the direction for −1 is perpendicular to it.

Figure 15: A qubit with real amplitudes, drawn in the plane.

-1: the outcomes are its eigenvalues. Measuring in the basis $|0\rangle$, $|1\rangle$, with $+1$ for 0 and -1 for 1, is the observable Z, and measuring along $|+\rangle$ and $|-\rangle$ is X, two of the *Pauli matrices*:

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle; \qquad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad X|+\rangle = |+\rangle, \quad X|-\rangle = -|-\rangle.$$

In the code, the outcomes $+1$ and -1 are `Plus` and `Minus`. The observable $-O$ is the same measurement as O, with the outcomes swapped. Every measurement in this paper has the form

$$\cos\varphi\, Z + \sin\varphi\, X$$

for an angle φ. Its eigenvector for $+1$ is the direction at half that angle, $[\cos\frac{\varphi}{2}, \sin\frac{\varphi}{2}]$, and its eigenvector for -1 is the perpendicular direction. Written out, the matrix is $\left(\begin{smallmatrix} \cos\varphi & \sin\varphi \\ \sin\varphi & -\cos\varphi \end{smallmatrix}\right)$: the reflection of the plane in the line at angle $\varphi/2$, which leaves that line fixed (eigenvalue $+1$) and turns the perpendicular line around (-1). We name an observable by φ, and give its directions, at $\varphi/2$, in degrees. Z is the case $\varphi = 0$ and points along $|0\rangle$; X is $\varphi = \pi/2$ and points along $|+\rangle$, at 45°. Bob's observables in Section 5.1, $(Z \pm X)/\sqrt{2}$, are $\varphi = \pm\pi/4$ and point at $\pm 22.5°$ (Figure 15b).[7]

Projectors and the Born rule. The part of $|\psi\rangle$ along $|v_+\rangle$ is $c_+|v_+\rangle$. The matrix that keeps this part and drops the other is the *projector* $P_+ = (I+O)/2$, where I is the identity matrix, and $P_- = (I-O)/2$ keeps the other part:

$$P_+|\psi\rangle = c_+|v_+\rangle, \qquad P_-|\psi\rangle = c_-|v_-\rangle, \qquad P_+ + P_- = I.$$

(Indeed $(I+O)/2$ maps $|v_+\rangle$ to $|v_+\rangle$, and $|v_-\rangle$ to 0.) The probability of an outcome is the squared length of the part that its projector P keeps, and the state that the outcome leaves is that part, divided by its length:

$$\langle\psi|P|\psi\rangle = c^2, \qquad \frac{P|\psi\rangle}{\sqrt{\langle\psi|P|\psi\rangle}},$$

where c is c_+ or c_-. Here $\langle\psi|P|\psi\rangle$ is the inner product of $|\psi\rangle$ with $P|\psi\rangle$, which is c^2 because $|v_+\rangle$ and $|v_-\rangle$ are perpendicular. The first formula is the *Born rule*, and the second the rule for the state after a

[7] In the experiments with photons of Section 1, such a measurement is a polarizer turned by the angle $\varphi/2$.

measurement; Section 5.1 calls both the Born rule, and `probability` and `collapse` compute them. An outcome with $c = 0$ never occurs, and leaves no state. The state that an outcome leaves matters only to a later measurement of the same qubit, or of the same pair; for a measurement that nothing follows, only the probability matters. That is why the local responses of Section 5.3 leave the state alone, and only `nonlocalA` and `nonlocalB`, each followed by the other box, collapse it.

An example. Measure $|0\rangle$ with X. Since $|0\rangle = (|+\rangle + |-\rangle)/\sqrt{2}$, the outcome is $+1$ or -1 with probability $1/2$ each, and the qubit is left in $|+\rangle$ or in $|-\rangle$. Measuring Z now gives $+1$ or -1 with probability $1/2$ each, although before the measurement of X it would have given $+1$ for sure. A state makes some measurements certain and leaves others to chance, and a measurement changes it.

Averages. The average outcome, $(+1)\langle\psi|P_+|\psi\rangle + (-1)\langle\psi|P_-|\psi\rangle$, is $\langle\psi|O|\psi\rangle$, since $P_+ - P_- = O$. For $|0\rangle$ it is 1 with Z, and 0 with X.

A.3 Two Qubits and Entanglement

Two bits have four values, 00, 01, 10 and 11, and a state of two qubits has an amplitude for each:

$$|\psi\rangle = \psi_{00}|00\rangle + \psi_{01}|01\rangle + \psi_{10}|10\rangle + \psi_{11}|11\rangle = [\psi_{00}, \psi_{01}, \psi_{10}, \psi_{11}],$$

with squares that add up to 1. The first digit belongs to Alice's qubit, and the second to Bob's. Measured in their basis, the two qubits give jk with probability ψ_{jk}^2. In the code, a state of the pair is this list of four numbers, a `Vector`.

Product states. If Alice's qubit is in the state $|u\rangle = [u_0, u_1]$ and Bob's, independently, in $|v\rangle = [v_0, v_1]$, the pair is in their *tensor product*

$$|u\rangle \otimes |v\rangle = [u_0v_0,\ u_0v_1,\ u_1v_0,\ u_1v_1].$$

Each amplitude is a product, so each probability is one too, $u_j^2\, v_k^2$: the two outcomes are independent. For example, $|00\rangle = |0\rangle \otimes |0\rangle$, and $|{+}{+}\rangle = |+\rangle \otimes |+\rangle = \frac{1}{2}[1, 1, 1, 1]$.

Entangled states. A state of the pair that is not a tensor product is *entangled*. The state $|\Phi^+\rangle$ of Section 5.2,

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2}}\,[1, 0, 0, 1],$$

is one. A product with $u_0v_0 \neq 0$ and $u_1v_1 \neq 0$ has $u_0 \neq 0$ and $v_1 \neq 0$, so also $u_0v_1 \neq 0$; but the second amplitude of $|\Phi^+\rangle$ is 0. Neither qubit of $|\Phi^+\rangle$ has a state vector of its own: as we show below, each gives each outcome with probability $1/2$ whatever is measured, and no unit vector $|u\rangle$ does that, since measuring along $|u\rangle$ itself gives $+1$ for sure. Only the pair has a state. Measured in their basis, its two qubits give 00 or 11, with probability $1/2$ each: their outcomes are always equal, and random.

Operators on the pair. A matrix that acts on the pair is four by four. The *tensor product* $M \otimes N$ of two two-by-two matrices acts on each qubit of a product state, $(M \otimes N)(|u\rangle \otimes |v\rangle) = M|u\rangle \otimes N|v\rangle$, and is the four-by-four matrix of blocks

$$M \otimes N = \begin{pmatrix} m_{00}\,N & m_{01}\,N \\ m_{10}\,N & m_{11}\,N \end{pmatrix},$$

which the code computes with the operator spelled >< and printed ⊗. A matrix $M \otimes I$ acts on Alice's qubit alone, and $I \otimes N$ on Bob's. For example, $Z \otimes I$ is the diagonal matrix with the entries $1, 1, -1, -1$: it maps $|jk\rangle$ to $\pm|jk\rangle$, with the sign set by Alice's digit j alone. (Applying an observable to a state is not measuring it; a measurement uses the projectors.)

Four rules of algebra. Section 5.3 computes with projectors on the pair by four rules.

(i) A matrix moves across an inner product as its transpose, $\langle Mu|v\rangle = \langle u|M^{\mathsf{T}}v\rangle$; a *symmetric* matrix, $M = M^{\mathsf{T}}$, moves unchanged. We write $\langle u|M|v\rangle$ for $\langle u|Mv\rangle$.

(ii) Every observable $O = \cos\varphi\, Z + \sin\varphi\, X$ is symmetric, and $OO = I$: a reflection done twice. Hence its projectors are symmetric, $P_{\pm}P_{\pm} = P_{\pm}$, and $P_{+}P_{-} = 0$.

(iii) The tensor product is linear in each argument, $(A \otimes B)(C \otimes D) = AC \otimes BD$, and $(A \otimes B)^{\mathsf{T}} = A^{\mathsf{T}} \otimes B^{\mathsf{T}}$. So $P \otimes I$ and $I \otimes Q$ are symmetric projectors too, they commute, and their product is $P \otimes Q$.

(iv) For a symmetric P with $PP = P$, $\langle\psi|P|\psi\rangle = \langle P\psi|P\psi\rangle$, the squared length of $P|\psi\rangle$.

By (iv), the rule of Section A.2 holds for every such projector, on one qubit or on the pair: an outcome has the squared length of the part that its projector keeps as its probability, and leaves that part, divided by its length.

Measuring one qubit of a pair. Alice measures her qubit with the projectors $P \otimes I$ of her observable, and Bob his with $I \otimes Q$; by rule (iv), the rule of Section A.2 applies to these four-by-four projectors unchanged. They are `measA` and `measB` in Section 5.1. For example, let Alice measure Z on $|\Phi^+\rangle$. Her projector for $+1$ is $P_+ = (I + Z)/2$, the diagonal matrix with the entries $1, 0$, so $P_+ \otimes I$ has the entries $1, 1, 0, 0$. It keeps the part $\frac{1}{\sqrt{2}}[1, 0, 0, 0]$ of $|\Phi^+\rangle$: her outcome is $+1$ with probability $1/2$, and it leaves the pair in $|00\rangle$. Bob's qubit is now $|0\rangle$, and his own Z gives $+1$ for sure. Her outcome -1 leaves $|11\rangle$ instead.

In general, let $|d\rangle = [d_0, d_1]$ be the direction of Alice's outcome, so that its projector on her qubit maps $|u\rangle$ to $\langle d|u\rangle\,|d\rangle$. On the pair it keeps

$$(P_d \otimes I)\,|\psi\rangle = |d\rangle \otimes |w\rangle, \qquad w_k = d_0\,\psi_{0k} + d_1\,\psi_{1k},$$

so her outcome has probability $\langle w|w\rangle$ and leaves Bob's qubit in $|w\rangle$, divided by its length. For $|\Phi^+\rangle$, $|w\rangle = |d\rangle/\sqrt{2}$: whatever Alice measures, each outcome has probability $1/2$. And her outcome leaves Bob's qubit in the direction that she found. (Equivalently, $|\Phi^+\rangle = (|d\rangle \otimes |d\rangle + |d_\perp\rangle \otimes |d_\perp\rangle)/\sqrt{2}$ for every direction $|d\rangle$ and its perpendicular $|d_\perp\rangle$, which needs real amplitudes.) This is the collapse of Section 5.2, and the dependence that the joint response of the quantum boxes carries from `nonlocalA` to `nonlocalB` (Section 5.3).

Less entangled pairs. The states $|\psi_\theta\rangle = \cos\theta\,|00\rangle + \sin\theta\,|11\rangle$ of Section 5.2 run from the product state $|00\rangle$ at $\theta = 0$ to $|\Phi^+\rangle$ at $\theta = \pi/4$. In between, the two qubits still give equal outcomes when both are measured in their basis, but 0 is more likely than 1. Here $|w\rangle = [d_0\cos\theta, d_1\sin\theta]$: when Alice measures Z, Bob's qubit is left in her direction, but when she measures X, it is left at the angle $\pm\theta$, not along $|\pm\rangle$. Only at $\theta = \pi/4$ is every measurement of either qubit on its own a fair coin; that is why $|\Phi^+\rangle$ is called *maximally* entangled.

A.4 Correlations, the CHSH Value, and No-Signaling

Joint probabilities. When Alice measures with the projectors P_x and Bob with Q_y, the pair of outcomes (x, y) has the probability

$$\langle\psi|P_x \otimes Q_y|\psi\rangle,$$

whichever of them measures first. Section 5.3 derives this from the rule of Section A.2; there P_x and Q_y stand for the projectors on the pair, $P_x \otimes I$ and $I \otimes Q_y$, whose product is $P_x \otimes Q_y$. The average of the product xy, the correlator E of Section 1, follows as the average outcome of Section A.2 did:

$$E = \sum_{x,y} xy\,\langle\psi|P_x \otimes Q_y|\psi\rangle = \langle\psi|(P_+ - P_-) \otimes (Q_+ - Q_-)|\psi\rangle = \langle\psi|A \otimes B|\psi\rangle$$

for Alice's observable A and Bob's observable B. This is the formula of Section 5.2, which `correlation` computes.

The CHSH value of $|\Phi^+\rangle$. Let Alice measure along the direction at the angle $\varphi_A/2$, and Bob along the one at $\varphi_B/2$. Alice's outcome leaves Bob's qubit in the direction that she found, so his outcome agrees with hers with probability $\cos^2\frac{\varphi_A-\varphi_B}{2}$, the squared component of her direction along his. Hence

$$E = \cos^2\frac{\varphi_A - \varphi_B}{2} - \sin^2\frac{\varphi_A - \varphi_B}{2} = \cos(\varphi_A - \varphi_B).$$

In the four contexts of Section 5.1, Alice measures along 0° and 45°, and Bob along 22.5° and −22.5° (Figure 15b). In three contexts their directions are 22.5° apart, and the outcomes agree with probability $\cos^2 22.5° \approx 0.854$; in `(A1, B1)` they are 67.5° apart, and agree with probability $\cos^2 67.5° \approx 0.146$. These are close to the frequencies of agreement in the table of Section 5.4. So E is $1/\sqrt{2}$ in three contexts and $-1/\sqrt{2}$ in the fourth, and $\text{CHSH} = 4/\sqrt{2} = 2\sqrt{2}$. No other directions do better. Let a_0, a_1, b_0, b_1 be the unit vectors at the angles φ of the four observables, so that $E = \cos(\varphi_A - \varphi_B)$ is the inner product of Alice's vector with Bob's. Regrouping as in Section 1, $\text{CHSH} = \langle a_0|b_0 + b_1\rangle + \langle a_1|b_0 - b_1\rangle$, which is at most the sum of the lengths of $b_0 + b_1$ and $b_0 - b_1$; their squared lengths add up to 4, so the sum is at most $2\sqrt{2}$. By Tsirelson's theorem, no other quantum state or measurement does better either (Tsirelson, 1980). The Bell table of Section 7.2 measures along 0° and 30° for Alice, and 0° and −30° for Bob: the directions are 0°, 30°, 30° and 60° apart, and the outcomes agree with probability 1, 3/4, 3/4 and 1/4.

No-signaling. Bob's outcome depends on Alice's outcome, and, given her outcome, on what she measured: after her Z gives +1 his Z is certain, and after her X gives +1 it is a fair coin. Averaged over her outcome, which Bob does not see, his qubit is left in her direction or in the perpendicular one, with probability 1/2 each. Whatever he measures, at an angle γ to her direction, he then

answers +1 with probability $\frac{1}{2}\cos^2\gamma + \frac{1}{2}\sin^2\gamma = \frac{1}{2}$. In general, Alice's two projectors add up to the identity, so summing the joint probability over her outcome leaves

$$\sum_x \langle\psi|P_x \otimes Q_y|\psi\rangle = \langle\psi|I \otimes Q_y|\psi\rangle,$$

in which her observable does not appear. This is the argument of Section 5.3.

Less entangled pairs. For $|\psi_\theta\rangle$, the averages of $Z \otimes Z$, $X \otimes X$, $Z \otimes X$ and $X \otimes Z$ are 1, $\sin 2\theta$, 0 and 0, so

$$E = \cos\varphi_A \cos\varphi_B + \sin 2\theta \ \sin\varphi_A \sin\varphi_B,$$

which gives the CHSH value $\sqrt{2}\,(1+\sin 2\theta)$ of Section 5.2 for the paper's observables. The argument for $|\Phi^+\rangle$, with Bob's vectors scaled by 1 along the first axis and by $\sin 2\theta$ along the second, bounds the CHSH value by $2\sqrt{1+\sin^2 2\theta}$, and other observables reach that bound; it exceeds $\sqrt{2}\,(1+\sin 2\theta)$ except at $\theta = \pi/4$. On its own, Alice's qubit gives +1 with probability $\cos^2\theta$ for Z and $1/2$ for X, and Bob's gives +1 with probability $\frac{1}{2}(1+\cos 2\theta/\sqrt{2})$ for either of his observables. At $\theta = \pi/8$ these are 0.854, 0.500 and 0.750, the probabilities that the run of Section 5.4 matches. Its table also has two cells at 0: after Alice's X gives ± 1, Bob's qubit is left at $\pm 22.5^\circ$, exactly the direction for +1 of `B0` or of `B1`, so there Bob answers `Plus` for sure.

Notion	In mathematics	In the code (Section 5)
a state of the pair	$[\psi_{00}, \psi_{01}, \psi_{10}, \psi_{11}]$	`Vector`, `maximallyEntangled`, `entangled`
inner product	$\langle u\|v\rangle$	`u <| v`
a matrix applied to a vector	$M\|v\rangle$	`m |> v`
tensor product	$M \otimes N$	`m ⊗ n`
observables	Z, X, $(Z \pm X)/\sqrt{2}$	`pauliZ`, `pauliX`, `observableA`, `observableB`
projectors	$P_\pm = (I \pm O)/2$	`projector`
projectors on the pair	$P \otimes I$, $I \otimes Q$	`measA`, `measB`
probability of an outcome	$\langle\psi\|P\|\psi\rangle$	`probability`
the state that it leaves	$P\|\psi\rangle/\sqrt{\langle\psi\|P\|\psi\rangle}$	`collapse`
an outcome, drawn	+1 with probability p	`coin`
correlator	$\langle\psi\|A \otimes B\|\psi\rangle$	`correlation`

Figure 16: The notions of this appendix, and their names in the code of Section 5.